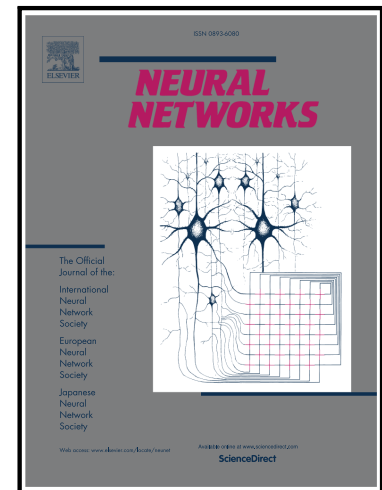


DATCNN: a novel CNN network with all the advantages of KAN while offering greater flexibility

Ruikun Luo, Nan Su, Yixiang Dai, Guosheng Li, Guijin Wang

PII: S0893-6080(26)01037-3
DOI: <https://doi.org/10.1016/j.neunet.2026.109580>
Reference: NN 109580



To appear in: *Neural Networks*

Received date: 10 July 2025
Revised date: 14 August 2026
Accepted date: 30 August 2026

Please cite this article as: Ruikun Luo, Nan Su, Yixiang Dai, Guosheng Li, Guijin Wang, DATCNN: a novel CNN network with all the advantages of KAN while offering greater flexibility, *Neural Networks* (2025), doi: <https://doi.org/10.1016/j.neunet.2026.109580>

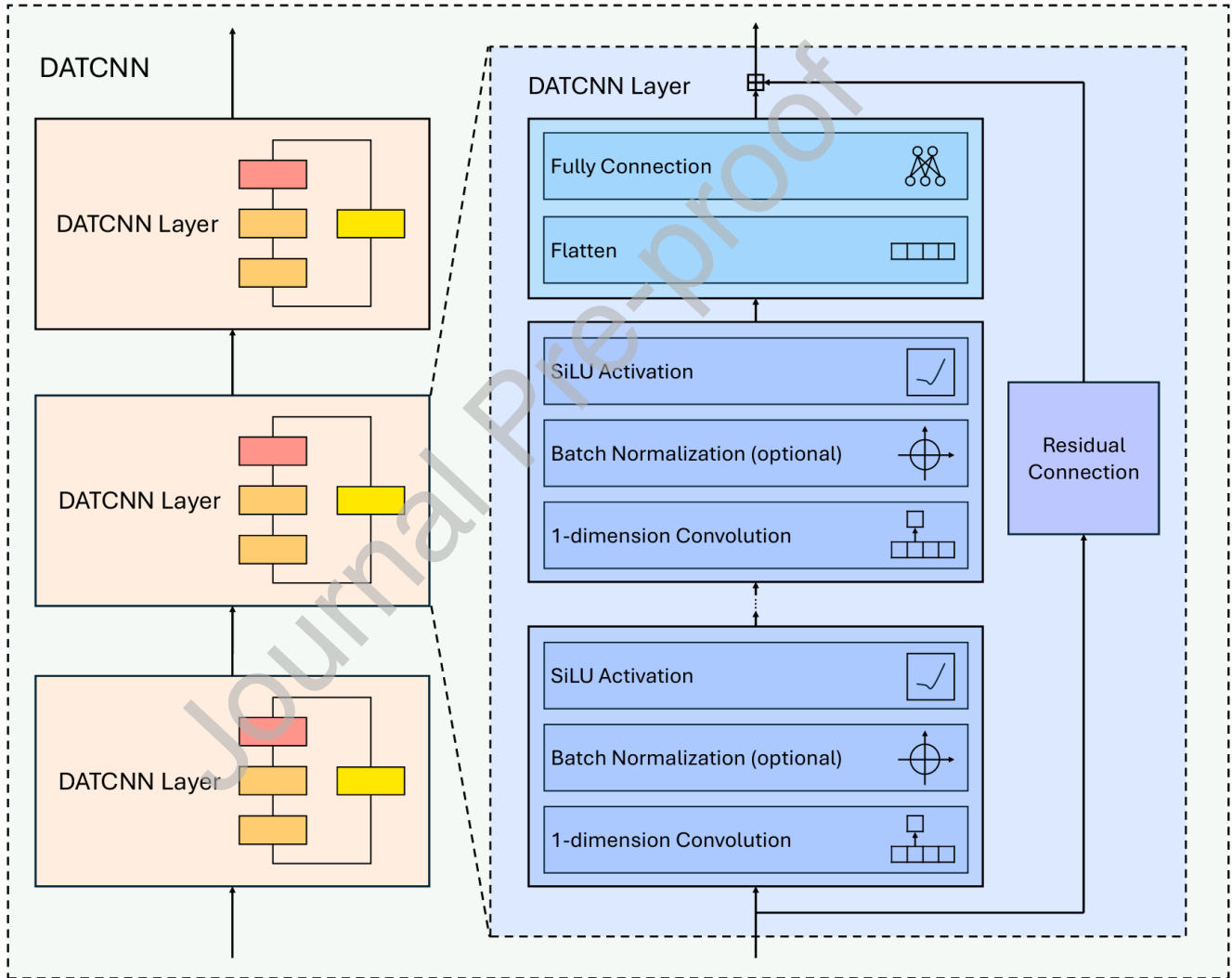
This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Graphical Abstract

DATCNN: a novel CNN network with all the advantages of KAN while offering greater flexibility

Ruikun Luo, Nan Su, Yixiang Dai, Guosheng Li, Guijin Wang

In recent years, the interpretability of artificial intelligence models has been increasingly valued. Kolmogorov–Arnold Network (KAN) is a novel neural network architecture that supports symbolic regression and offers good interpretability. Derivative works of KAN not only provide interpretability but also demonstrate good performance in common tasks such as image segmentation and time series prediction. However, KANs suffer from slow convergence and difficult training processes. To address these limitations, we propose the Dense Affine Transformation Convolutional Neural Network (DATCNN). This novel CNN-based architecture preserves the interpretability and functional representation capacity of KANs while offering enhanced flexibility and compatibility with established CNN theory and training heuristics. Experimental results across multiple tasks, including function fitting, image classification, and natural language processing, demonstrate that DATCNN achieves faster training speeds and superior performance, highlighting its potential as an efficient alternative to KANs in theoretical and practical settings.



Highlights

DATCNN: a novel CNN network with all the advantages of KAN while offering greater flexibility

Ruikun Luo, Nan Su, Yixiang Dai, Guosheng Li, Guijin Wang

- We propose DATCNN, a novel convolutional neural network architecture that inherits the core strengths of KAN while offering significantly enhanced flexibility.
- We formulate a mathematical theory for DATCNN, theoretically guaranteeing its capabilities in symbolic regression and interpretability.
- Extensive experiments demonstrate that DATCNN outperforms KAN and other state-of-the-art variants, achieving superior accuracy with reduced training latency.

DATCNN: a novel CNN network with all the advantages of KAN while offering greater flexibility

Ruikun Luo^{a,1}, Nan Su^{a,1}, Yixiang Dai^a, Guosheng Li^a, Guijin Wang^{a,b,*}

^a*Department of Electronic Engineering, Tsinghua University, Beijing, 100084, China*

^b*Shanghai AI Laboratory, Shanghai, 200232, China*

Abstract

In recent years, the interpretability of artificial intelligence models has been increasingly valued. Kolmogorov–Arnold Network (KAN) is a novel neural network architecture that supports symbolic regression and offers good interpretability. Derivative works of KAN not only provide interpretability but also demonstrate good performance in common tasks such as image segmentation and time series prediction. However, KANs suffer from slow convergence and difficult training processes. To address these limitations, we propose the Dense Affine Transformation Convolutional Neural Network (DATCNN). This novel CNN-based architecture preserves the interpretability and functional representation capacity of KANs while offering enhanced flexibility and compatibility with established CNN theory and training heuristics. Experimental results across multiple tasks, including function fitting, image classification, and natural language processing, demonstrate that DATCNN achieves faster training speeds and superior performance, highlighting its potential as an efficient alternative to KANs in theoretical and practical settings.

Keywords: XAI, CNN

1. Introduction

The Kolmogorov–Arnold representation theorem demonstrates that any bounded multivariate continuous function can be expressed as a finite composition of univariate functions and addition, thereby providing theoretical guidance for the representational capacity of neural networks [1]. Multilayer perceptrons (MLPs) [1, 2, 3] are among the fundamental building blocks of contemporary deep neural networks. MLPs have given rise to numerous advanced network architectures such as convolutional neural networks (CNNs) [4, 5] and transformers [6], all of which have found extensive applications in fields like computer vision and natural language processing. However, these models are typically characterized by their complex and nested structures, which result in poor interpretability. The potential faults within these models are challenging to detect in advance. In some scenarios, they may lead to catastrophic consequences, posing legal and ethical risks [7]. In recent years, researchers have proposed numerous methods related to model interpretability, which can be categorized into four types based on the format of the generated explanations [7]: examples, attributions, hidden semantics, and rules. Example-based explanations provide additional supporting or refuting examples for any new input, such as the most similar examples or representative sample sets within each category [8, 9]. Attribution-based explanations indicate the influence of each input feature on the output,

for instance, saliency maps [10, 11, 12, 13]. Explanations based on hidden semantics focus on the meanings of hidden neurons or layers, attempting to link abstract concepts with the activation of specific hidden neurons [14, 10]. Whether it is examples, attributions, or hidden semantics, the explanations for models are indirect. In contrast, rule-based explanations offer the most direct explanations by providing a set of rules or decision trees equivalent to the model [15, 16]. However, the representational capacity of rule sets and decision trees is limited, making it challenging to describe mathematical functions. To overcome this drawback, Symbolic regression methods have gradually replaced rule-based methods and become the mainstream of explanation solutions, as they possess stronger representational capabilities and can provide mathematical expressions [17, 18]. Nevertheless, most existing symbolic regression methods are search-based and directly target end-to-end data, resulting in a rapidly growing search space as the model size increases, making it difficult to handle complex models.

The Kolmogorov–Arnold Network (KAN) [19] is a novel network architecture that supports symbolic regression and offers good interpretability. KAN is composed of many learnable activation functions. During symbolic regression, KAN can search for each activation function separately, avoiding the vast search space problem. Despite the promising potential of KAN in many scenarios, slow convergence and difficult training have always been significant drawbacks. Due to the use of learnable activation functions, KAN exhibits an essential structural difference

*Corresponding author, Email: wangguijin@tsinghua.edu.cn

¹Equal Contribution.

from MLPs and their derivatives, such as CNNs and transformers. As a result, KAN lacks flexibility, meaning that existing methods for accelerating model training are challenging to transfer directly to KAN, and the effectiveness of any transferred methods lacks both theoretical and empirical guarantees.

In recent years, researchers have proposed new learnable activation functions to improve the training speed of KAN [20, 21, 22]. fKAN [20] is the most advanced variant of KAN, replacing the ordinary activation functions in MLP with trainable Jacobian basis functions. However, the performance of learnable activation functions is directly related to the form of the ground truth. Each learnable activation function has its specific application scenarios and does not have universal advantages. Moreover, these methods still have the drawback of insufficient flexibility. PowerMLP [23] is a distinct variant of KAN that employs fixed higher-order functions, ReLU^k , as activation layers within an MLP. While this modification reduces computational overhead, it incurs a performance trade-off. In addition to modifying the learnable activation functions of KAN, researchers have also widely applied KAN to various common tasks, such as image segmentation [24, 25], image classification [26], time series prediction [27], and graph problems [28, 29]. These approaches can be summarized as either inserting KAN as an independent module into the model or replacing the MLP module in the original model with KAN, thereby helping the model better distinguish highly nonlinear local latent concepts [24]. For example, Convolutional KAN [26] replaces the generalized linear module in convolutional layers with KAN, thereby improving image classification performance. However, this also brings KAN's drawbacks into the models, leading to slower training speeds and reduced flexibility.

In this paper, we propose a novel convolutional neural network structure, called the Dense Affine Transformation Convolutional Neural Network (DATCNN), to address the lack of uniformity between KAN and mainstream CNN architectures. Each DATCNN layer employs a dual-path architecture comprising a backbone and a residual connection. Its backbone is formed by stacking multiple convolutional layers with batch normalization and activation, followed by a fully connected layer without activation. By applying affine transformations to activation functions within the convolutional layers, DATCNN achieves the effect of learnable activation functions in KAN, thereby facilitating symbolic regression and enhancing interpretability. Compared to KAN, DATCNN offers superior flexibility by effectively leveraging established CNN theories and empirical design principles. Overall, DATCNN preserves the interpretability benefits of KAN while providing greater flexibility.

We theoretically validate the DATCNN architecture through mathematical analysis and simulation experiments. Its performance is comprehensively evaluated across diverse scenarios, including function fitting, image classification,

and natural language processing. Experimental results demonstrate that DATCNN outperforms KAN and its variants in both accuracy and training efficiency, highlighting its flexibility and broad application potential. Our key contributions are summarized as follows.

- We propose DATCNN, a novel convolutional neural network architecture that inherits the core strengths of KAN while offering significantly enhanced flexibility.
- We formulate a mathematical theory for DATCNN, theoretically guaranteeing its capabilities in symbolic regression and interpretability.
- Extensive experiments demonstrate that DATCNN outperforms KAN and other state-of-the-art variants, achieving superior accuracy with reduced training latency.

2. Kolmogorov-Arnold Networks

The inspiration for KAN originates from the Kolmogorov-Arnold representation theorem, which states that any bounded multivariate continuous function can be expressed as a finite composition of univariate functions and addition:

$$f(x_1, x_2, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right). \quad (1)$$

KAN observes that the right-hand side of Equation 1 can be viewed as stacking two layers, thereby generalizing the above representation theorem to wider and deeper cases. Figure 1 illustrates the structure of a single KAN Layer. Specifically, a single KAN Layer Φ_l can be regarded as a matrix composed of several univariate functions:

$$\Phi_l = \begin{bmatrix} \phi_{l,1,1} & \phi_{l,1,2} & \cdots & \phi_{l,1,n_l} \\ \phi_{l,2,1} & \phi_{l,2,2} & \cdots & \phi_{l,2,n_l} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,n_{l+1},1} & \phi_{l,n_{l+1},2} & \cdots & \phi_{l,n_{l+1},n_l} \end{bmatrix}, \quad (2)$$

And the input-output relationship of the KAN Layer can be expressed in the form of matrix multiplication:

$$\begin{aligned} \Phi_l(x) &= \begin{bmatrix} \phi_{l,1,1} & \phi_{l,1,2} & \cdots & \phi_{l,1,n_l} \\ \phi_{l,2,1} & \phi_{l,2,2} & \cdots & \phi_{l,2,n_l} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,n_{l+1},1} & \phi_{l,n_{l+1},2} & \cdots & \phi_{l,n_{l+1},n_l} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n_l} \end{bmatrix} \\ &= \begin{bmatrix} \sum_{p=1}^{n_l} \phi_{l,1,p}(x_p) \\ \sum_{p=1}^{n_l} \phi_{l,2,p}(x_p) \\ \vdots \\ \sum_{p=1}^{n_l} \phi_{l,n_{l+1},p}(x_p) \end{bmatrix}. \end{aligned} \quad (3)$$

A typical KAN is a stacking of several KAN Layers. For example, an L-layer KAN can be expressed as:

$$\text{KAN}(x) = (\Phi_L \circ \Phi_{L-1} \circ \cdots \circ \Phi_1)(x). \quad (4)$$

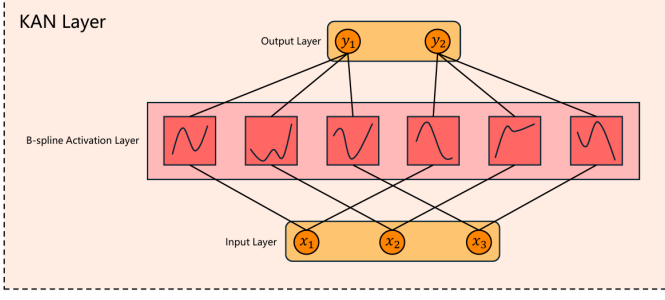


Figure 1: The structural diagram of a KAN Layer. A KAN Layer with input dimension d_I and output dimension d_O has a total of $d_I d_O$ learnable activation functions. Each output is formed by summing the inputs after each passes through a learnable activation function.

The ability of KAN to learn activation functions directly is implemented by the B-spline:

$$\phi(x) = wb \cdot \text{SiLU}(x) + ws \cdot \sum_{\nu=1}^n c_{\nu} B_{\xi,p,\nu}(x). \quad (5)$$

where wb, ws, c_{ν} are weights, and $B_{\xi,p,\nu}$ is a set of functions known as B-splines. Their definition is recursive, as shown in Definition 1 [30].

Definition 1. For any natural number p and an increasing finite real sequence ξ (length $n + p + 1$), a set of real functions $B_{\xi,p,\nu}$ can be recursively defined as:

$$B_{\xi,p,\nu}(x) = \frac{x - \xi_{\nu}}{\xi_{\nu+p} - \xi_{\nu}} B_{\xi,p-1,\nu}(x) + \frac{\xi_{\nu+p+1} - x}{\xi_{\nu+p+1} - \xi_{\nu+1}} B_{\xi,p-1,\nu+1}(x), \quad (6)$$

$$B_{\xi,0,\nu}(x) = \begin{cases} 1 & x \in [\xi_{\nu}, \xi_{\nu+1}), \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

The sequence ξ in Definition 1 is called the knot vector, an essential parameter of KAN. However, in KAN, the optimization of ξ is based on rules rather than gradient descent, which is one of the main reasons for the slow convergence of KAN. Another important reason is the high computational complexity of B-splines themselves. It can be seen that B-splines provide KAN with the ability to learn activation functions directly; however, they also cause KAN to exhibit slow convergence and difficult training, leaving much room for improvement.

To facilitate subsequent mathematical arguments, we formally define the KAN Layer as shown in Definition 2.

Definition 2. For any natural number p , finite real sequence ξ (length $n + p + 1$), and real tensors wb, ws (shape (d_O, d_I)), c (shape (d_O, d_I, n)), the KAN Layer is a mapping from $\mathbb{R}^{d_I}$ to $\mathbb{R}^{d_O}$:

$$\begin{aligned} & [\text{KANLayer}(x; \xi, p, wb, ws, c)]_i \\ &= \sum_{j=0}^{d_I-1} \phi_{ij}(x_j; \xi, p, wb_{ij}, ws_{ij}, c_{ij}), \end{aligned} \quad (8)$$

where:

$$\begin{aligned} \phi_{ij}(x_j; \xi, p, wb_{ij}, ws_{ij}, c_{ij}) \\ = wb_{ij} \cdot \text{SiLU}(x_j) + ws_{ij} \cdot \text{spline}_{ij}(x_j; \xi, p, c_{ij}), \end{aligned} \quad (9)$$

$$\text{spline}_{ij}(x_j; \xi, p, c_{ij}) = \sum_{\nu=0}^{n-1} c_{ij\nu} B_{\xi,p,\nu}(x_j). \quad (10)$$

3. Method

This section presents DATCNN and elucidates its theoretical underpinnings. As illustrated below, DATCNN is derived from a novel convolutional neural network architecture equivalent to KAN, incorporating specific enhancements. Section 3.1 provides a detailed structure of DATCNN. Section 3.2 analyzes the translational symmetry of B-splines in KAN. Building on Section 3.2, Section 3.3 defines a new function called the normalized B-function, which makes the computation of B-splines more convenient and flexible. Further, Section 3.3 transforms the normalized B-function into a non-recursive form and provides a method for calculating the coefficients. Section 3.4 offers a novel perspective on the findings in Section 3.3 and designs a convolutional network structure equivalent to KAN. Section 3.5 refines this architecture while preserving its symbolic regression capabilities, culminating in the proposal of DATCNN in Section 3.1.

3.1. Dense Affine Transformation Convolutional Neural Network

The proposed DATCNN consists of several stacked DATCNN layers, as shown in Figure 2. A DATCNN layer employs a dual-path structure, consisting of a backbone and a residual connection. Its backbone is formed by stacking multiple convolutional layers with batch normalization and activation, followed by a fully connected layer without activation. When the input and output dimensions are the same, the residual path uses the identity mapping; when they differ, it uses a fully connected layer. In practice, all batch normalization and the activation layers in the residual path are optional. Definition 3 provides a formal description of the DATCNN layer.

Definition 3. For input dimension d_I and output dimension d_O , the DATCNNLayer is a mapping from $\mathbb{R}^{d_I}$ to $\mathbb{R}^{d_O}$, as shown in Equation (11), where FC denotes a fully connected layer, Act denotes an activation layer, BatchNorm denotes a batch normalization layer, Conv denotes a convolutional layer, and Residual denotes a residual connection:

$$\begin{aligned} & \text{DATCNNLayer}(x) \\ &= \text{FC} \circ \text{Act} \circ \text{BatchNorm} \circ \text{Conv} \circ \dots \circ \text{Act} \\ & \quad \circ \text{BatchNorm} \circ \text{Conv}(x) + \text{Residual}(x). \end{aligned} \quad (11)$$

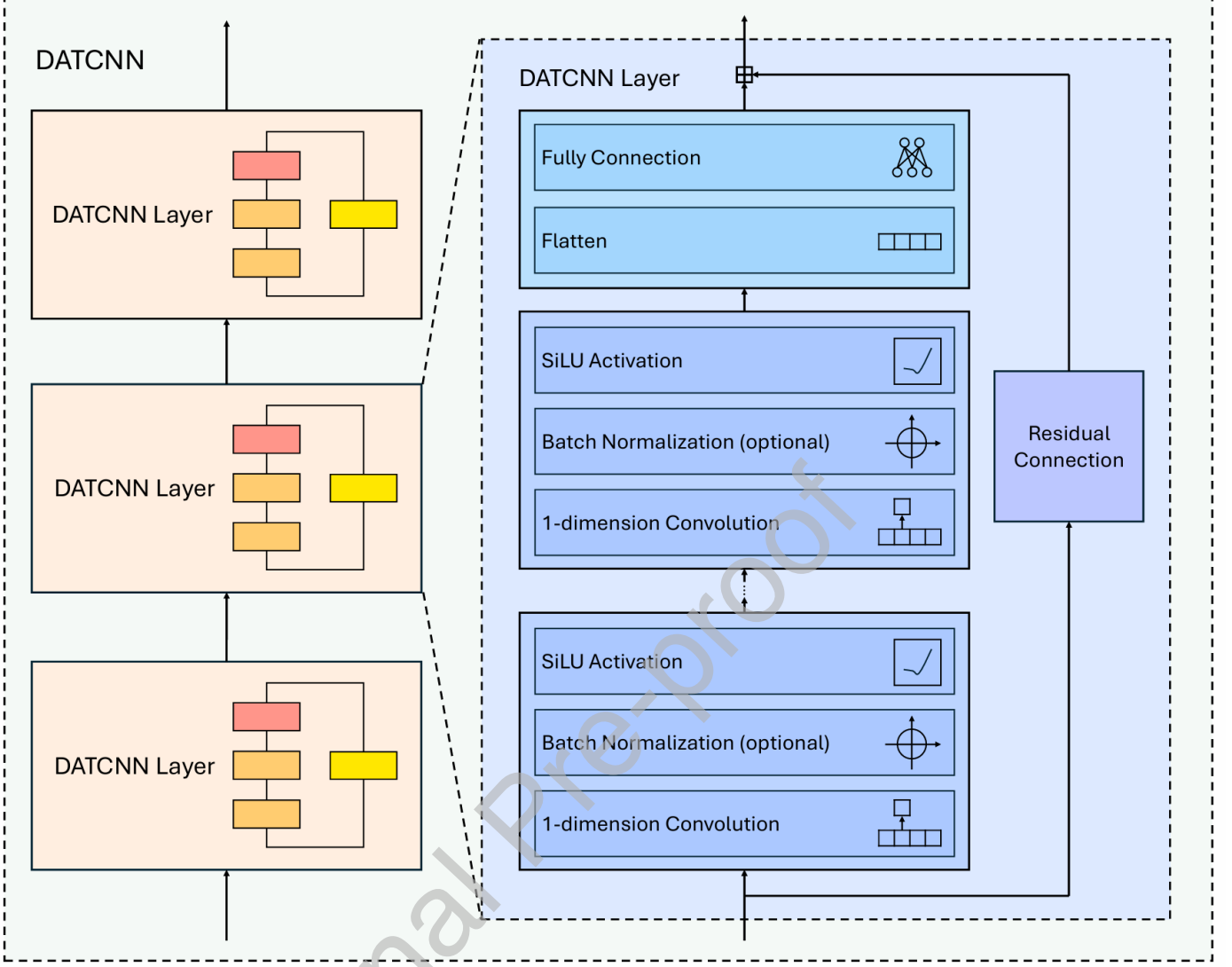


Figure 2: Structural diagram of DATCNN. A DATCNN layer uses a dual-path structure, consisting of a backbone and a residual connection. Its backbone is formed by stacking multiple convolutional layers with batch normalization and activation, followed by a fully connected layer without activation at the end.

It is worth noting that the convolution in DATCNN is typically dense, that is, the number of convolutional kernels is significantly greater than the dimensions of both the input and the output. The composition of convolution and activation functions can be viewed as an affine transformation of the activation functions. Numerous activation functions transformed by affine transformations can be the basis for learnable activation functions, thereby enabling DATCNN to possess symbolic regression capabilities.

3.2. Properties of B-splines

Figure 3 illustrates B-splines, where Figure 3a shows a general B-spline, and Figure 3b shows a special case with uniformly spaced knots. In this special case, the shapes of $B_{\xi,p,\nu}$ for different indices ν are identical, differing only in position. This conclusion can be formally stated and proven by Theorem 1.

Theorem 1. For a B-spline with parameters p and ξ (length $n + p + 1$), if ξ is uniform, i.e.,

$$\forall \nu \in \mathbb{Z} \cap [0, n - 1] \quad \xi_\nu = \xi_0 + \nu\delta, \quad (12)$$

then:

$$\forall \nu \in \mathbb{Z} \cap [0, n - 1] \quad B_{\xi,p,\nu}(x) = B_{\xi,p,0}(x - \nu\delta). \quad (13)$$

PROOF. We use induction on p . When $p = 0$, the statement is true. Assume it holds for some natural number p ,

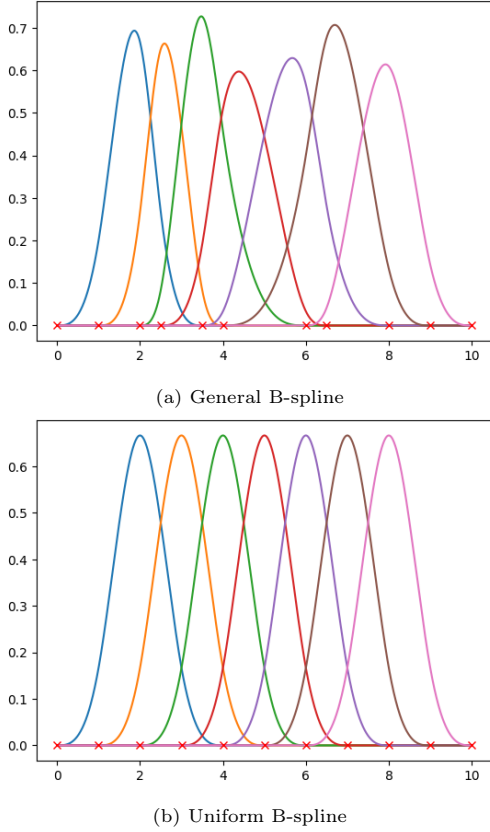


Figure 3: Illustration of B-splines. Figure 3a shows a general B-spline, and Figure 3b shows a special case with uniformly spaced knots, where the shapes of $B_{\xi,p,\nu}$ for different indices ν are identical, differing only in position.

and consider the case for $p+1$:

$$\begin{aligned}
 B_{\xi,p+1,\nu}(x) &= \frac{x - \xi_\nu}{\xi_{\nu+p+1} - \xi_\nu} B_{\xi,p,\nu}(x) + \frac{\xi_{\nu+p+2} - x}{\xi_{\nu+p+2} - \xi_{\nu+1}} B_{\xi,p,\nu+1}(x) \\
 &= \frac{x - \nu\delta - \xi_0}{(p+1)\delta} B_{\xi,p,0}(x - \nu\delta) \\
 &\quad + \frac{\xi_0 + (p+2)\delta - (x - \nu\delta)}{(p+1)\delta} B_{\xi,p,0}(x - \nu\delta - \delta),
 \end{aligned} \tag{14}$$

Substituting $\nu = 0$ gives:

$$\begin{aligned}
 B_{\xi,p+1,0}(x) &= \frac{x - \xi_0}{(p+1)\delta} B_{\xi,p,0}(x) \\
 &\quad + \frac{\xi_0 + (p+2)\delta - x}{(p+1)\delta} B_{\xi,p,0}(x - \delta),
 \end{aligned} \tag{15}$$

It is easy to see that $B_{\xi,p+1,\nu}(x) = B_{\xi,p+1,0}(x - \nu\delta)$.

Although strict translational symmetry only holds for uniform knots, Figure 3a shows that even when the knots are not uniform, the components of the B-spline are still similar smooth pulse curves. One component can be approximately transformed into another by horizontal stretching and translation, as well as vertical scaling, which can

be implemented using the convolutional and fully connected layers in DATCNN. In the subsequent mathematical derivations, we only discuss the case of uniform knots, but the conclusion we obtain is also approximately valid for non-uniform knots. The correctness of this generalization will be verified in the experimental section.

3.3. Normalized B-function

To compute B-splines more efficiently, we define the normalized B-function as the 0th B-function when $\xi_n = n$, as shown in Definition 4. With the help of Proposition 2, we present a method to represent B-splines using the normalized B-function, as shown in Theorem 3.

Definition 4. The definition of the p -th order normalized B-function is:

$$\hat{B}_p(x) = B_{\xi,p,0}(x) \quad \text{where} \quad \hat{\xi}_\nu = \nu. \tag{16}$$

Proposition 2. The p -th order B-function satisfies the following recursive relationship:

$$\hat{B}_p(x) = \frac{x}{p} \hat{B}_{p-1}(x) + \frac{p+1-x}{p} \hat{B}_{p-1}(x-1), \tag{17}$$

$$\hat{B}_0(x) = \begin{cases} 1 & \text{if } x \in [0, 1), \\ 0 & \text{otherwise.} \end{cases} \tag{18}$$

PROOF. Substituting $\xi_\nu = \nu$ and $\nu = 0$ into Definition 1 yields the result.

Theorem 3. If ξ is uniform, then the B-spline can be expressed using the normalized B-function. Let the interval be δ , then:

$$B_{\xi,p,\nu}(x) = \hat{B}_p\left(\frac{x - \xi_0}{\delta} - \nu\right). \tag{19}$$

PROOF. For Definition 1, taking $\xi_\nu = \xi_0 + \nu\delta$ and $\nu = 0$, and replacing x with $\delta x + \xi_0$, we obtain:

$$\begin{aligned}
 B_{\xi,p,0}(\delta x + \xi_0) &= \frac{x}{p} B_{\xi,p-1,0}(\delta x + \xi_0) \\
 &\quad + \frac{p+1-x}{p} B_{\xi,p-1,0}[\delta(x-1) + \xi_0],
 \end{aligned} \tag{20}$$

$$B_{\xi,0,0}(\delta x + \xi_0) = \begin{cases} 1 & \text{if } x \in [0, 1), \\ 0 & \text{otherwise.} \end{cases} \tag{21}$$

It can be seen that $B_{\xi,p,0}(\delta x + \xi_0) = \hat{B}_p(x)$, since their recursive formulas are identical. Using Theorem 1 and replacing x with $\frac{x - \xi_0}{\delta} - \nu$ completes the proof.

Theorem 3 is significant because it shows that any B-spline with arbitrary parameters can be represented using a deterministic normalized B-function. This decouples the recursive structure in the definition of B-splines from the knot points, thereby allowing for simplified computation. Proposition 4 further provides a method for computing the normalized B-function.

Proposition 4. The normalized B-function $\hat{B}_p$ can be expressed as a piecewise function:

$$\hat{B}_p(x) = \begin{cases} A_{p,0}(x) & \text{if } x \in [0, 1), \\ A_{p,1}(x) & \text{if } x \in [1, 2), \\ \dots & \dots \\ A_{p,p}(x) & \text{if } x \in [p, p+1). \end{cases} \quad (22)$$

Here, $A_{p,\mu}$ is a polynomial of degree p :

$$A_{p,\mu}(x) = \frac{x}{p} A_{p-1,\mu}(x) + \frac{p+1-x}{p} A_{p-1,\mu-1}(x-1), \quad (23)$$

$$A_{0,0}(x) = 1. \quad (24)$$

PROOF. We use induction on p . When $p = 0$, the statement is true. Assume it holds for some natural number p , and consider the case for $p+1$. Substituting into Theorem 3 completes the proof.

The coefficients of each term in $A_{p,\mu}$ are independent of the input and can be computed once during function instantiation. For example, $\hat{B}_p$ can be represented as a piecewise quadratic polynomial:

$$\hat{B}_2(x) = \begin{cases} \frac{1}{2}x^2 & x \in [0, 1), \\ -x^2 + 3x - \frac{3}{2} & x \in [1, 2), \\ \frac{1}{2}x^2 - 3x + \frac{9}{2} & x \in [2, 3). \end{cases} \quad (25)$$

Therefore, $\hat{B}_p$ remains fixed during training and inference, thereby reducing computational cost.

3.4. A Convolutional Network Architecture Equivalent to KAN

Theorem 3 demonstrates that a normalized B-function can be transformed into a B-spline through scaling and translation. As indicated by the right-hand side of Equation (19), these operations can be implemented via a one-dimensional convolution with a kernel size of 1. Consequently, a convolutional network architecture can replicate the computation graph of KAN. We denote this proposed architecture as DATCNNKAN. As illustrated in Figure 4, the model consists of stacked DATCNNKAN layers. The backbone of each layer comprises three components: a one-dimensional convolution, a normalized B-function activation, and a fully connected layer. Additionally, the layer includes a residual connection comprising a SiLU activation and a fully connected layer. The formal definition is provided in Definition 5, and its equivalence to KAN is rigorously established in Theorem 5.

Definition 5. For any real tensors wsc (shape (d_O, nd_I)) and wb (shape (d_O, d_I)), an integer p , and real numbers ξ_0 and δ , the DATCNNKAN layer is a mapping from $\mathbb{R}^{d_I}$ to $\mathbb{R}^{d_O}$:

$$\begin{aligned} & \text{DATCNNKANLayer}(x; wb, wsc, p, \xi_0, \delta) \\ &= \text{FC}_{wsc} \circ \hat{B}_p \circ \text{Conv1}_{\xi_0, \delta}(x) + \text{FC}_{wb} \circ \text{SiLU}(x), \end{aligned} \quad (26)$$

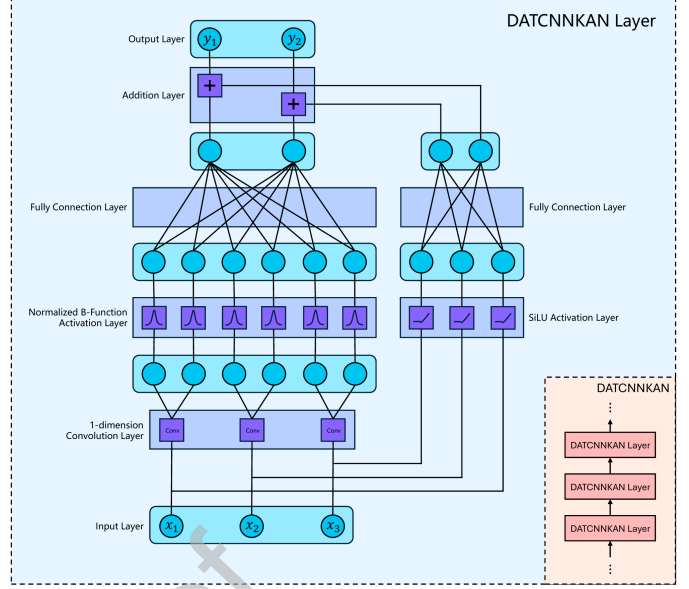


Figure 4: Structural diagram of DATCNNKAN. DATCNNKAN is composed of multiple stacked DATCNNKAN layers. The backbone of a DATCNNKAN layer consists of three parts: a one-dimensional convolution layer with kernel size 1, a normalized B-function activation layer, and a fully connected layer. In addition to the backbone, the DATCNN layer also includes a residual connection, which is composed of a SiLU activation layer and a fully connected layer.

here, FC denotes a fully connected layer:

$$[\text{FC}_w(x)]_i = \sum_j w_{ij} x_j, \quad (27)$$

and Conv1 denotes a one-dimensional convolution with kernel size 1:

$$[\text{Conv1}_{\xi_0, \delta}(x)]_{\nu j} = \frac{x_j}{\delta} - \left(\nu + \frac{\xi_0}{\delta} \right). \quad (28)$$

Theorem 5. For a KAN Layer with uniform knots, let the order of the B-spline be p , and the knot points be:

$$\xi_\nu = \xi_0 + \nu\delta, \quad (29)$$

then there exists a DATCNNKAN layer that is equivalent to it:

$$\begin{aligned} & \text{KANLayer}(x; \xi, p, wb, ws, c) \\ &= \text{DATCNNKANLayer}(x; wb, wsc, p, \xi_0, \delta), \end{aligned} \quad (30)$$

here:

$$wsc = \text{reshape}(ws \otimes c, [d_O, nd_I]). \quad (31)$$

PROOF. According to Definitions 2 and 5, and Theorem

3:

$$\begin{aligned}
& [\text{KANLayer}(x; \xi, p, wb, ws, c)]_i \\
&= \sum_{j=0}^{d_I-1} \phi_{ij}(x_j; \xi, p, wb_{ij}, ws_{ij}, c_{ij}) \\
&= \sum_{j=0}^{d_I-1} \left[wb_{ij} \text{SiLU}(x_j) + ws_{ij} \sum_{\nu=0}^{n-1} c_{ij\nu} B_{\xi, p, \nu}(x_j) \right] \\
&= \sum_{j=0}^{d_I-1} wb_{ij} \text{SiLU}(x_j) + \sum_{j=0}^{d_I-1} \sum_{\nu=0}^{n-1} ws_{ij} c_{ij\nu} \hat{B}_p \left(\frac{x_j - \xi_0}{\delta} - \nu \right) \quad (32) \\
&= \text{FC}_{wb} \circ \text{SiLU}(x) + \text{FC}_{wsc} \circ \hat{B}_p \circ \text{Conv1}_{\xi_0, \delta}(x) \\
&= \text{DATCNKANLayer}(x; wb, wsc, p, \xi_0, \delta).
\end{aligned}$$

3.5. From DATCNKAN to DATCNN

Although the DATCNKAN architecture exhibits a certain degree of complexity, the convolutional, activation, and fully connected layers in its backbone are indispensable, as they directly ensure both symbolic regression capability and interpretability. To enhance the model while preserving its symbolic regression capabilities, we have refined DATCNKAN in several key aspects. First, we replace the normalized B-function with the SiLU activation function, which offers superior derivative properties. Second, inspired by ResNet [5], we streamline the residual connections to facilitate gradient propagation: an identity mapping is used when the input and output dimensions match, whereas a single fully connected layer is used when they differ. Third, to stabilize training, we apply batch normalization after the convolutional layers in the main branch and after the fully connected layers in the residual path. Finally, the convolutional layers in the main branch can be stacked. These enhancements culminate in the DATCNN architecture described in Section 3.1.

Compared with KAN, DATCNN introduces several key architectural modifications. First, it eliminates KAN's grid structure, replacing its functionality with convolutional layers to enable gradient-based optimization. Second, the parameterized recursive form of the original B-splines is replaced with a fixed SiLU activation, significantly reducing computational costs. Furthermore, the dual-path architecture of DATCNN includes batch normalization, the SiLU activation function, and residual connections to improve gradient stability. With these structural changes, DATCNN preserves the core advantages of KAN, specifically its symbolic regression capability and high interpretability.

Table 1: Relative Errors of B-spline

δ	ξ_0	p	n	Relative Error $\downarrow$
0.4	-2	1	5	6.3e-31
0.1	-1.5	3	20	1.5e-29
0.02	-1.2	5	100	2.5e-26
0.005	-1.2	10	500	7.2e-18

Table 2: Relative Errors between KAN Layer and DATCNKAN Layer

d_I	d_O	p	n	Relative Error $\downarrow$
1	1	1	5	0
3	2	3	20	1.0e-31
10	5	5	100	3.6e-29
100	20	10	500	5.3e-18

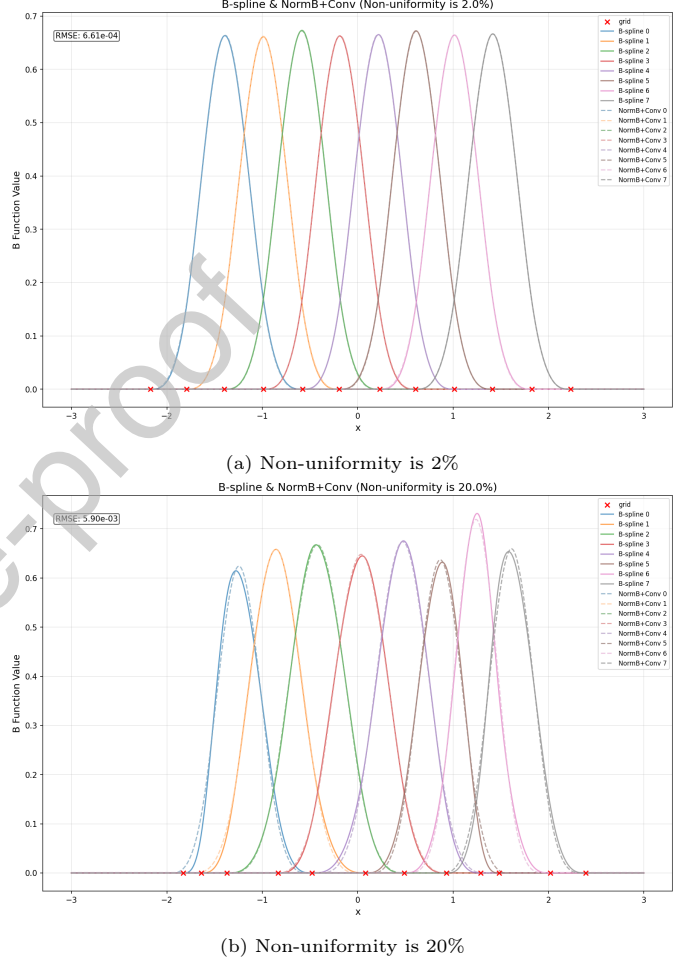


Figure 5: Illustration of approximation error in the case of a non-uniform grid. Figure 5a shows the case with 2% non-uniformity, and Figure 5b shows the case with 20% non-uniformity.

4. Experiment

The experiment consists of three parts. In Section 4.1, we verify Theorems 3 and 5 by numerical simulation to check the correctness of the theoretical derivation. In Section 4.2, following the experimental design in the KAN paper, we compare the function fitting capabilities of KAN and DATCNN. In Section 4.3.1, we evaluate the effectiveness of DATCNN in common image classification and natural language processing tasks, demonstrating its flexibility and application value.

Table 3: Function Fitting Results on Feynman and Special Function Datasets

Dataset	Model	Avg. RMSE ↓	Std. RMSE ↓	Avg. Time(s) ↓	Std. Time(s) ↓
Feynman	KAN	<u>0.0032</u>	<u>0.0138</u>	639	473
	fKAN	0.0225	0.1954	403	153
	PowerMLP	0.0084	0.0368	128	48
	DATCNN	0.0025	0.0043	<u>152</u>	<u>102</u>
Special Function	KAN	0.0027	<u>0.0124</u>	448	360
	fKAN	0.0180	0.2297	474	148
	PowerMLP	0.0467	0.1200	124	34
	DATCNN	<u>0.0034</u>	0.0096	<u>174</u>	<u>135</u>

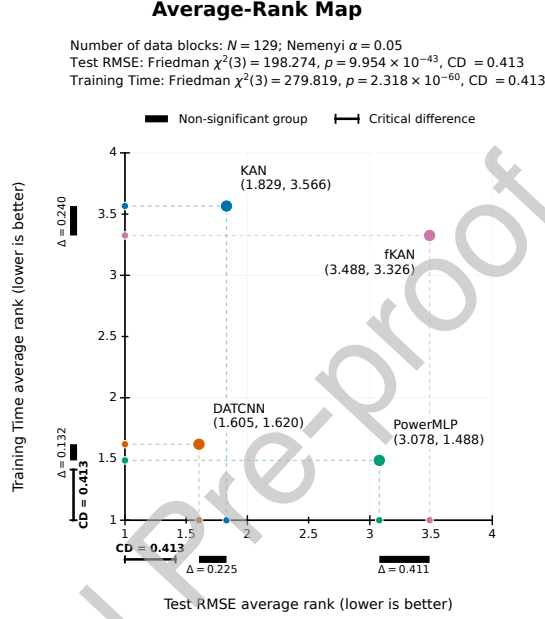


Figure 6: Two-metric average-rank map for test RMSE and training time on the combined Feynman and special-function datasets. Lower average ranks are better. Capped rulers show the Nemenyi critical difference (CD) at $\alpha = 0.05$, and thick bars connect models without significant pairwise differences.

Table 4: Ablation study on Feynman I.8.4.

model	Depth of Conv. Layer	Activation Function	BatchNorm	RMSE ↓	Training Time (s) ↓
KAN	-	-	-	0.0056	570
DATCNN	1	NormB	False	0.0050	202
	1	NormB	True	0.0039	186
	1	SiLU	True	0.0025	169
	2	SiLU	True	0.0023	240

4.1. Simulation Validation of Mathematical Theory

We first verify Theorem 3. For B-splines with different uniform grids and different orders, we compute both sides of Equation (19) using Definition 1 and Proposition 4, respectively. We use a random array of length 100,000 as input and then compare the relative errors. The results on both sides of the equation are arrays of shape $100,000 \times n$, and the relative error is defined as the ratio of the mean squared error between the two arrays to the sum of squares of a single array.

Table 1 presents the relative error between the left and right sides of formula (12). In the table header, δ , ξ_0 , and n are the parameters of the uniform grid, as detailed in Equation (12), and p is the order of the B-spline, as defined in Definition 1. Although the relative error gradually increases with the increase of p and n , it remains within the rounding error range, confirming the correctness of Theorem 3.

Next, we verify Theorem 5. For different numbers of grids, orders of B-spline, and layer shapes in the layer,

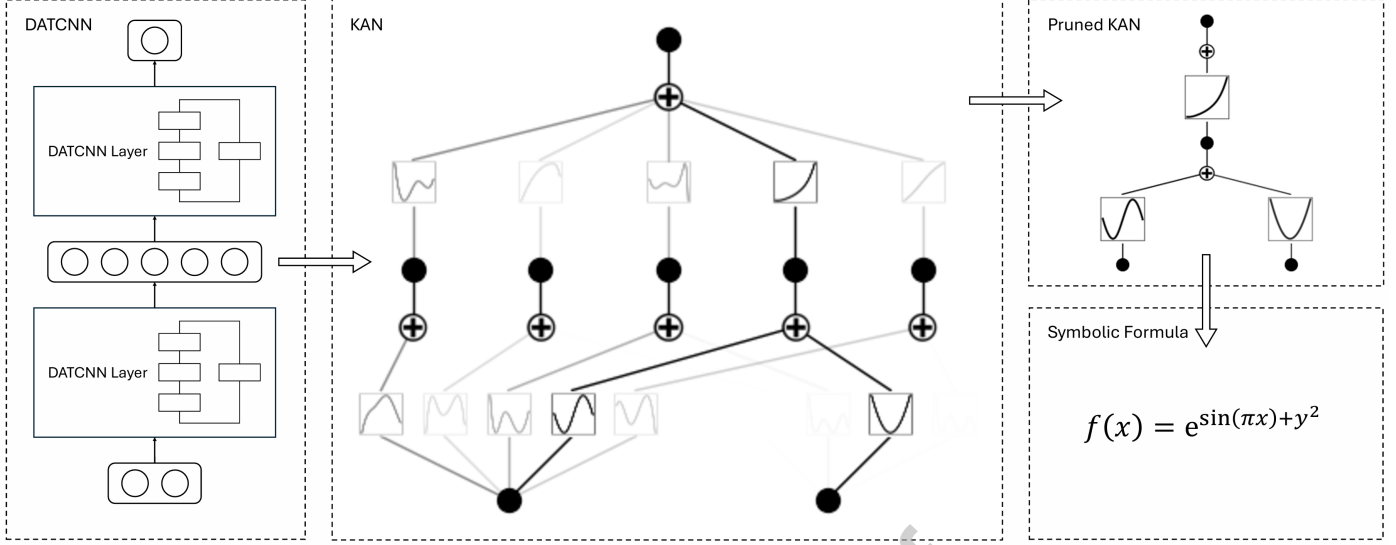


Figure 7: A demonstration of symbolic regression: First, train a DATCNN on a dataset; then extract the backbone parameters of the DATCNN, interpret them as learnable activation functions, and convert the model into a KAN; finally, follow KAN’s symbolic regression workflow.

Table 5: Results of the hyperparameter sensitivity experiments on Feynman I.6.20b. Each group varies one hyperparameter while keeping the others at the baseline configuration. Lower RMSE is better. Light-gray cells indicate relatively stable regimes for multi-valued hyperparameters or the lower-RMSE setting in each binary comparison.

Network Width		Network Depth		Conv. Depth		Learning Rate		Residual Connection		BatchNorm	
Value	RMSE ↓	Value	RMSE ↓	Value	RMSE ↓	Value	RMSE ↓	Value	RMSE ↓	Value	RMSE ↓
1	0.1062	2	0.1255	1	0.0017	10^{-4}	1.0624	True	0.0012	True	0.0013
2	0.0610	3	0.0059	2	0.0017	10^{-3}	0.0154	False	0.0014	False	0.0027
3	0.0015	4	0.0039	3	0.0012	10^{-2}	0.0091	—	—	—	—
4	0.0017	5	0.0016	4	0.0017	10^{-1}	0.0016	—	—	—	—
5	0.0010	6	0.0016	5	0.0588	0.3	0.0011	—	—	—	—
6	0.0018	7	0.0018	—	—	1	0.0016	—	—	—	—
7	0.0022	8	0.0014	—	—	2	0.0216	—	—	—	—

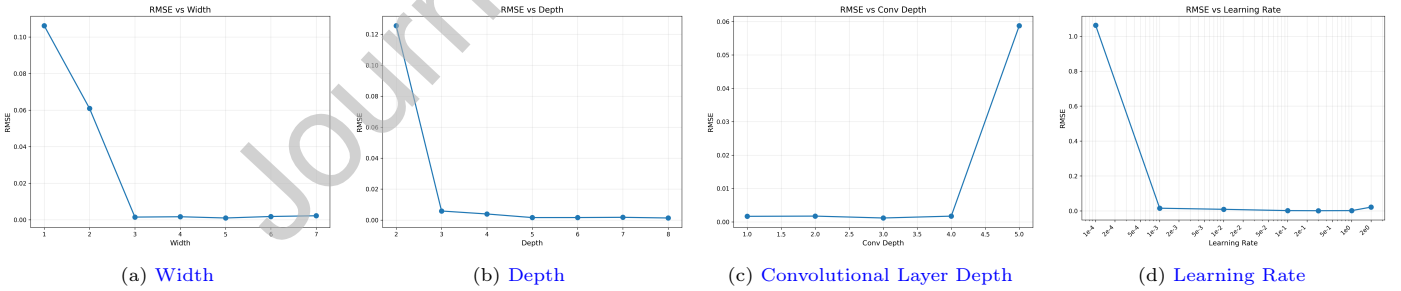


Figure 8: Experiments on hyperparameter sensitivity, including network width, network depth, convolutional layer depth, and learning rate.

we first instantiate a KAN Layer, migrate the parameters from the KAN Layer to the DATCNNKAN Layer, and then compute both sides of Equation (30) using Definitions 2 and 5, respectively. We use a random array of shape $1000 \times d_I$ (d_I is the input dimension of the layer) as input and compare their relative errors. The results on both sides of Equation (30) are arrays of shape $1000 \times d_O$ (d_O is the output dimension of the layer), and the relative error is defined as the ratio of the mean squared error between the two arrays to the sum of squares of a single array.

Table 2 presents the relative error between the left and right sides of formula (30). In the table header, d_I and d_O represent the input and output dimensions of the KAN Layer or DATCNNKAN Layer, as detailed in Definitions 2 and 5, p is the order of the B-spline, as defined in Definition 1, and n is the parameter of the uniform grid, as detailed in Equation (12). Although the relative error gradually increases with the increase of d_I, d_O, p, n , it always remains within the range of rounding errors, confirming the correctness of Theorem 5.

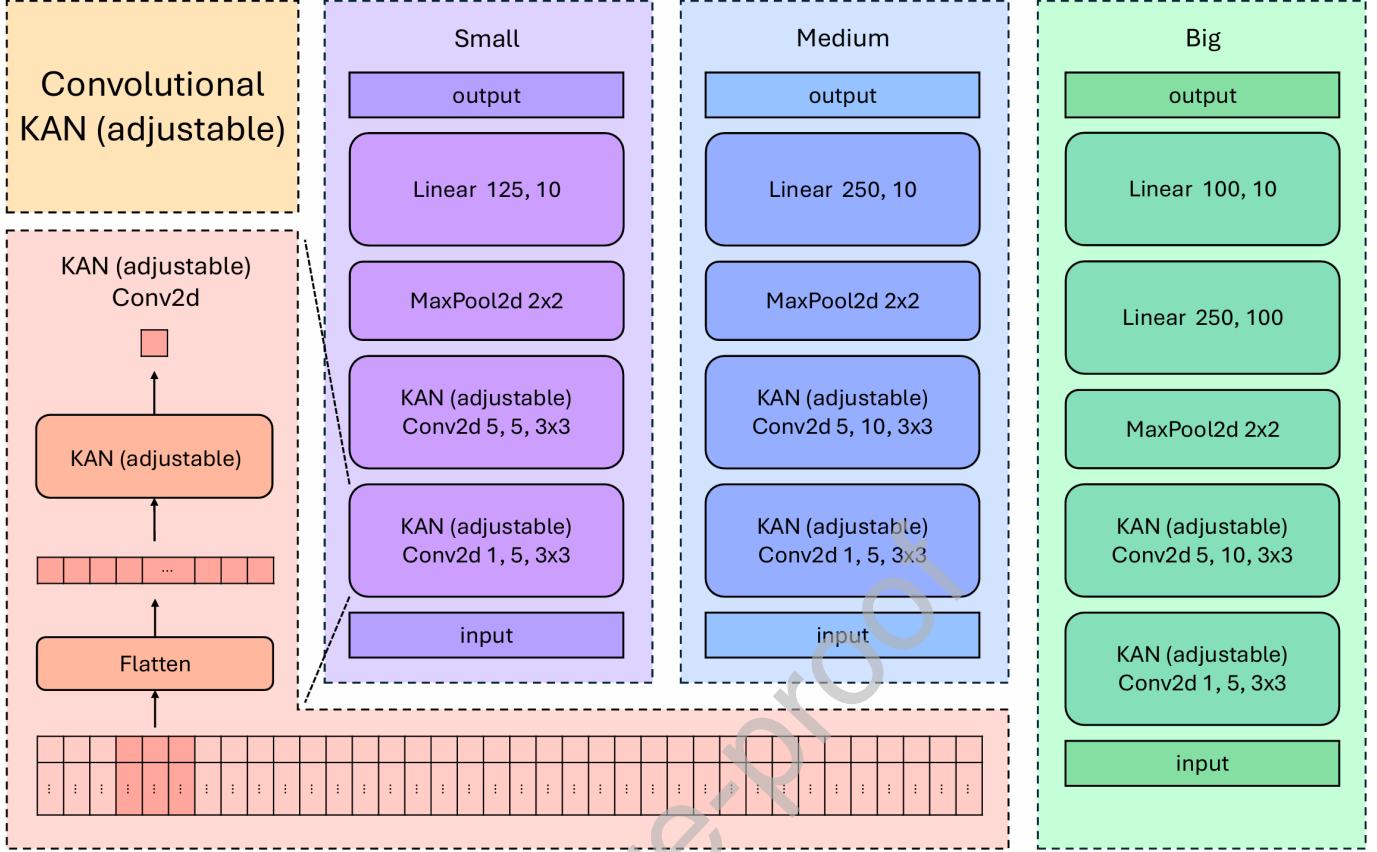


Figure 9: The structural diagram of Convolutional KAN. Convolutional KAN is available in three sizes, all of which feature a similar structure. The core part is the KAN Conv2d Layer, located at the bottom left of the figure. Like a regular convolutional layer, the KAN Conv2d Layer has a rectangular window that slides over the input. However, instead of performing a dot product with the convolutional kernel, the elements within the window are flattened and passed through a KAN Layer to obtain the output.

We evaluated the approximation error between B-splines and normalized B-functions on non-uniform grids. In the standard KAN implementation, the default grid non-uniformity is set to 2%. At this level, the approximation error (measured by RMSE) is approximately $6.61e-4$. As illustrated in Figure 5a, the discrepancy between the two curves is visually negligible. Even when grid non-uniformity is increased to 20%, the approximation error remains minimal at $5.90e-3$ (see Figure 5b), with the curves retaining close alignment. These results empirically demonstrate that the mathematical conclusions derived in Section 3 can be verified, when grid non-uniformity does not exceed 20%.

4.2. Function Fitting

This part of the experiment compares the performance of KAN and DATCNN in function-fitting tasks. Following the experimental design outlined in the KAN paper [19], we utilize the special functions datasets [19] and the Feynman datasets [18]. The special function dataset compiles common special functions in mathematics and physics, whereas the Feynman dataset gathers physics equations from Feynman’s textbooks. The input and output dimensions of the model are adapted to the data, while all other

intermediate dimensions are set to 5. We optimize for 200 steps in each stage using an LBFGS optimizer with a learning rate of 1. We search for the optimal configuration within the model depth range of $\{2, 3, 4, 5, 6\}$. To enable a fair comparison with fKAN [20] and PowerMLP [23], the DATCNN and KAN are trained without grid extension.

Table 3 summarizes the function fitting results on the Feynman and special functions datasets. Figure 6 complements the table with a two-metric average-rank map. For each retained function, the four models are ranked separately by test RMSE and training time, and the horizontal and vertical coordinates are the corresponding ranks averaged over the 129 function blocks; lower ranks indicate better performance. For each metric, the Friedman test assesses whether the models differ overall, while the Nemenyi critical difference (CD) identifies pairwise differences: a rank gap larger than the CD is statistically significant at $\alpha = 0.05$, whereas models joined by a thick bar do not differ significantly.

Taken together, Table 3 and Figure 6 show that DATCNN and KAN do not differ significantly in test RMSE, whereas DATCNN significantly outperforms PowerMLP and fKAN. In training time, DATCNN and Pow-

Table 6: Image Classification Results on Fashion-MNIST and SVHN Datasets

Dataset	Size	Kernel	Accuracy $\uparrow$	Precision $\uparrow$	Recall $\uparrow$	F1 Score $\uparrow$	Params $\downarrow$	Training Time (s/epoch) $\downarrow$
Fashion MNIST	Small	KAN	0.8816	0.8814	0.8816	0.8814	15300	68
		PowerMLP	0.8791	0.8782	0.8791	0.8781	19290	40
		fKAN	<u>0.8860</u>	<u>0.8853</u>	<u>0.8860</u>	<u>0.8850</u>	11280	85
		DATCNN	0.8876	0.8868	0.8876	0.8870	14790	<u>47</u>
	Medium	KAN	0.8880	0.8867	0.8880	0.8866	28250	107
		PowerMLP	0.8917	0.8919	0.8917	0.8916	35565	55
		fKAN	<u>0.8937</u>	<u>0.8944</u>	<u>0.8937</u>	<u>0.8938</u>	20880	130
		DATCNN	0.8946	0.8950	0.8946	0.8942	27315	<u>69</u>
	Big	KAN	<u>0.9027</u>	<u>0.9029</u>	<u>0.9027</u>	<u>0.9026</u>	51850	107
		PowerMLP	0.8986	0.8983	0.8986	0.8981	59165	55
		fKAN	0.8996	0.9005	0.8996	0.8999	44480	130
		DATCNN	0.9067	0.9061	0.9067	0.9060	50915	<u>69</u>
SVHN	Small	KAN	0.7924	0.7765	0.7680	0.7708	20530	106
		PowerMLP	0.8442	0.8315	0.8284	<u>0.8294</u>	25850	53
		fKAN	<u>0.8533</u>	<u>0.8391</u>	0.8385	<u>0.8385</u>	15170	93
		DATCNN	0.8551	0.8478	<u>0.8359</u>	0.8411	19850	<u>60</u>
	Medium	KAN	0.8317	0.8181	0.8113	<u>0.8139</u>	34030	151
		PowerMLP	0.8526	0.8376	0.8378	0.8369	42675	73
		fKAN	<u>0.8642</u>	<u>0.8531</u>	<u>0.8493</u>	<u>0.8507</u>	25320	131
		DATCNN	0.8716	0.8625	0.8585	0.8600	32925	<u>82</u>
	Big	KAN	0.8634	0.8518	0.8499	0.8504	67530	152
		PowerMLP	0.8756	0.8647	0.8643	0.8642	76175	75
		fKAN	<u>0.8841</u>	0.8767	<u>0.8704</u>	0.8729	58820	138
		DATCNN	0.8844	<u>0.8715</u>	0.8759	<u>0.8727</u>	66425	<u>82</u>

Table 7: Ablation study on FashionMNIST. The model size is small.

kernel	Size of Conv. Kernel	Depth of Conv. Layer	Activation Function	F1 Score $\uparrow$	Params $\downarrow$	Training Time (s/epoch) $\downarrow$
KAN	-	-	-	0.8814	15300	68
DATCNN	1	1	NormB	0.8749	4890	53
	1	1	SiLU	0.8731	4890	41
	1	2	SiLU	0.8774	8190	45
	3	2	SiLU	0.8870	14790	47

erMLP do not differ significantly, whereas DATCNN is significantly faster than KAN and fKAN. Table 3 further shows that DATCNN has the lowest RMSE standard deviation, indicating superior stability. Consequently, DATCNN offers the most favorable trade-off between accuracy and efficiency.

Comprehensive results are detailed in Appendix A due to space constraints. Table 4 presents an ablation study of DATCNN on Formula I.8.4 from the Feynman dataset, validating the effectiveness of improvements in convolutional layer depth, activation functions, and batch normalization.

We analyzed the hyperparameter sensitivity of DATCNN. Starting from a baseline configuration (network width of 5, depth of 6, convolutional depth of 2, and a learning rate of 1, incorporating residual connections and batch normalization), we adjusted each hyperparameter individually. Using the Feynman dataset (formula I.6.20b) as a

case study, we monitored fluctuations in the fitting error. The complete numerical results are summarized in Table 5, while the four multi-valued hyperparameters are illustrated in Figure 8. DATCNN exhibits stability across a broad range of hyperparameter settings, thereby validating the rationale behind our experimental design.

Compared with improved variants such as fKAN and PowerMLP, DATCNN retains KAN’s symbolic regression capability and interpretability. Figure 7 gives an example of symbolic regression: to uncover the functional form underlying a dataset, we first fit the data using DATCNN, then extract its backbone parameters, interpret them as learnable activation functions, convert them into a KAN, and finally follow KAN’s symbolic regression pipeline. This approach combines the advantages of DATCNN’s fast training with the strong interpretability of symbolic systems.

Table 8: Text Classification on TREC-6 and TREC-50 Datasets

Dataset	Model	Accuracy $\uparrow$	Precision $\uparrow$	Recall $\uparrow$	F1 Score $\uparrow$	Training Time (s) $\downarrow$
TREC-6	KAN	0.9260	0.9366	0.9364	0.9353	5985
	fKAN	0.9600	0.9686	0.9496	0.9583	101
	PowerMLP	0.9520	0.9601	<u>0.9599</u>	<u>0.9594</u>	24
	DATCNN	<u>0.9560</u>	<u>0.9626</u>	0.9628	0.9625	<u>31</u>
TREC-50	KAN	<u>0.7200</u>	0.6057	0.5335	0.5175	16839
	fKAN	0.5180	0.3735	0.4138	0.3365	123
	PowerMLP	0.7120	<u>0.6302</u>	<u>0.6098</u>	<u>0.5839</u>	25
	DATCNN	0.7500	0.6645	0.6302	0.6186	<u>32</u>

Table 9: Semantic Similarity Regression on STS-B Dataset

Model	MAE $\downarrow$	RMSE $\downarrow$	Pearson $\uparrow$	Spearman $\uparrow$	Training Time (s) $\downarrow$
KAN	1.0800	1.2992	0.5527	0.5610	19562
fKAN	0.5697	0.7155	0.8801	0.8950	191
PowerMLP	<u>0.3762</u>	<u>0.5046</u>	<u>0.9420</u>	<u>0.9432</u>	40
DATCNN	0.2725	0.3478	0.9759	0.9759	<u>67</u>

4.3. Common Task

This section validates DATCNN’s performance on common tasks in image classification and natural language processing, demonstrating its flexibility and practical value.

4.3.1. Image Classification

This section evaluates the effectiveness of DATCNN in common image classification tasks. Convolutional KAN [26] is the first work to apply KAN to image classification, with a structure as shown in Figure 9. Convolutional KAN is available in three sizes, all of which feature a similar structure. The core part is the KAN Conv2d Layer, at the bottom left of Figure 9. Similar to a regular convolutional layer, the KAN Conv2d Layer has a rectangular window that slides over the input. However, instead of performing a dot product with the convolutional kernel, the elements within the window are flattened and passed through a KAN Layer to obtain the output. In this experiment, we retained the framework structure of Convolutional KAN, replaced the KAN layer with DATCNN layers, fKAN, and PowerMLP. To follow the experiments of Convolutional KAN [26] and PowerMLP [23], we selected the Fashion-MNIST [31] and the Street View House Numbers (SVHN) [32] datasets. Fashion-MNIST is a dataset containing images of Zalando’s products, which are associated with labels from 10 categories. SVHN contains over 600,000 labeled digits cropped from Street View images. All hyperparameters were set to the original settings of Convolutional KAN, that is, a hyperparameter search was conducted within the range of learning rates $\{1 \times 10^{-3}, 5 \times 10^{-4}, 1 \times 10^{-4}\}$, weight decay $\{0, 1 \times 10^{-5}, 1 \times 10^{-4}\}$, batch sizes $\{128, 64, 32\}$, and a maximum of 20 training epochs.

Table 6 summarizes the comparative results, demonstrating that DATCNN possesses significant advantages

in both classification performance and training efficiency. Furthermore, Table 7 presents an ablation study on the small-sized network. These results validate the efficacy of refinements in convolutional kernel size, layer depth, and activation functions, underscoring DATCNN’s flexibility and broad application potential.

4.3.2. Natural Language Processing

This section evaluates the performance of DATCNN on natural language processing tasks using the TREC-6, TREC-50 [33], and STS-B [34, 35] benchmarks. Derived from the TREC QA Track [36], TREC-6 and TREC-50 classify questions into six coarse and 50 fine-grained semantic categories, respectively. These datasets comprise 5,450 training, 500 validation, and 500 test samples. The STS-B dataset assesses semantic textual similarity between sentence pairs (5,750 training, 1,500 validation, 1,380 test samples) by requiring the model to predict a similarity score ranging from 0 (no overlap) to 5 (equivalence). Performance is evaluated based on the correlation between model predictions and human judgments. Regarding input representation, we employ averaged GloVe embeddings [37] for classification tasks and sentence-transformer embeddings [38] for regression tasks. Consistent with the function-fitting experiments, we set the model width to 5 and the depth search space to 2-6. All models are trained for 10 epochs using the Adam optimizer with a learning rate of 1×10^{-3} .

Tables 8 and 9 summarize the experimental results for the natural language processing tasks. Notably, DATCNN achieves superior performance while maintaining high training efficiency. In contrast, KAN’s training time is orders of magnitude longer than that of other models. This discrepancy is attributed to the 300-dimensional embedding used in this experiment, which is significantly higher than

the input dimensions in prior studies. These findings reveal inherent limitations in KAN’s computational graph, leading to a drastic surge in computational costs as the input dimension increases.

5. Conclusion

In this paper, we presented DATCNN, a novel convolutional neural network architecture that effectively integrates the core advantages of KAN. By refining a convolutional structure that is theoretically equivalent to KAN, DATCNN preserves KAN’s interpretability and symbolic regression capabilities while significantly enhancing architectural flexibility. We provided a theoretical justification for the proposed architecture via mathematical analysis and validated its effectiveness through extensive simulations. Empirical evaluations across diverse scenarios, including function fitting, image classification, and natural language processing, demonstrate that DATCNN outperforms KAN and its variants in both predictive performance and training efficiency. These results highlight the broad applicability of DATCNN as a flexible, interpretable deep learning backbone.

Appendix A. Result of Function Fitting Experiments

Table A.10: Result of Feynman Dataset

Num.	Function	Model	RMSE	Training Time(s)
I.6.20a	$\frac{1}{\sqrt{2\pi}}e^{-\frac{\theta^2}{2}}$	KAN	0.0004	672.8
		fKAN	0.0006	611.0
		PowerMLP	0.0065	85.4
		DATCNN	0.0004	24.5
I.6.20	$\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{\theta^2}{2\sigma^2}}$	KAN	0.0004	227.0
		fKAN	0.0011	479.9
		PowerMLP	0.0149	103.8
		DATCNN	0.0012	69.8
I.6.20b	$\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(\theta-\theta_1)^2}{2\sigma^2}}$	KAN	0.0027	258.2
		fKAN	0.1412	348.6
		PowerMLP	0.0260	115.6
		DATCNN	0.0012	211.5
I.8.4	$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$	KAN	0.0056	570.9
		fKAN	0.4974	412.0
		PowerMLP	0.0464	146.8
		DATCNN	0.0023	240.3
I.9.18	$\frac{Gm_1m_2}{(x_2-x_1)^2+(y_2-y_1)^2+(z_2-z_1)^2}$	KAN	0.0014	826.0
		fKAN	0.0278	254.0
		PowerMLP	0.0022	39.0
		DATCNN	0.0018	106.7
I.10.7	$\frac{m_0}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	0.0090	285.0
		fKAN	0.3535	491.2
		PowerMLP	0.0040	89.3
		DATCNN	0.0018	78.0
I.11.19	$x_1y_1 + x_2y_2 + x_3y_3$	KAN	0.0034	1934.8
		fKAN	0.0071	637.3
		PowerMLP	0.0038	153.3
		DATCNN	0.0028	208.1
I.12.1	μN_n	KAN	0.0005	758.5
		fKAN	0.0013	429.8
		PowerMLP	0.0015	65.9
		DATCNN	0.0009	109.5
I.12.2	$\frac{q_1q_2}{4\pi\epsilon r^2}$	KAN	0.0047	734.2
		fKAN	0.0017	365.1
		PowerMLP	0.0033	102.2
		DATCNN	0.0022	150.0
I.12.4	$\frac{q_1}{4\pi\epsilon r^2}$	KAN	0.0009	173.4
		fKAN	0.0625	242.9
		PowerMLP	0.0063	108.0
		DATCNN	0.0013	51.9
I.12.5	q_2E_f	KAN	0.0005	741.4
		fKAN	0.0013	356.3
		PowerMLP	0.0015	64.7
		DATCNN	0.0009	107.2
I.12.11	$q(E_f + Bv \sin(\theta))$	KAN	0.0974	1788.8
		fKAN	0.0104	295.2
		PowerMLP	0.1541	203.5
		DATCNN	0.0097	392.9

Num.	Function	Model	RMSE	Training Time(s)
I.13.4	$\frac{1}{2}m(v^2 + u^2 + w^2)$	KAN	<u>0.0014</u>	348.9
		fKAN	<u>0.0047</u>	538.4
		PowerMLP	<u>0.0017</u>	<u>147.8</u>
		DATCNN	0.0011	80.2
I.13.12	$Gm_1m_2 \left(\frac{1}{r_2} - \frac{1}{r_1} \right)$	KAN	<u>0.0022</u>	285.0
		fKAN	<u>0.0051</u>	263.0
		PowerMLP	<u>0.0033</u>	157.0
		DATCNN	0.0019	<u>174.7</u>
I.14.3	mgz	KAN	0.0007	234.2
		fKAN	<u>0.0095</u>	472.9
		PowerMLP	<u>0.0018</u>	64.0
		DATCNN	<u>0.0014</u>	<u>118.9</u>
I.14.4	$\frac{1}{2}k_s x^2$	KAN	0.0004	604.5
		fKAN	<u>0.0008</u>	<u>66.1</u>
		PowerMLP	<u>0.0008</u>	66.7
		DATCNN	<u>0.0009</u>	50.3
I.15.3x	$\frac{x-ut}{\sqrt{1-\frac{u^2}{c^2}}}$	KAN	<u>0.0304</u>	748.6
		fKAN	<u>0.0657</u>	506.8
		PowerMLP	<u>0.0210</u>	107.6
		DATCNN	0.0083	<u>326.1</u>
I.15.3t	$\frac{t-\frac{ux}{c^2}}{\sqrt{1-\frac{u^2}{c^2}}}$	KAN	<u>0.0356</u>	960.7
		fKAN	<u>0.0494</u>	469.0
		PowerMLP	<u>0.0315</u>	204.1
		DATCNN	0.0141	<u>215.2</u>
I.15.10	$\frac{m_0 v}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	<u>0.0029</u>	337.5
		fKAN	<u>0.0051</u>	<u>118.8</u>
		PowerMLP	0.0026	112.9
		DATCNN	<u>0.0030</u>	125.3
I.16.6	$\frac{u+v}{1+\frac{uv}{c^2}}$	KAN	0.0013	295.1
		fKAN	<u>0.0039</u>	140.5
		PowerMLP	<u>0.0042</u>	<u>127.3</u>
		DATCNN	<u>0.0028</u>	90.9
I.18.4	$\frac{m_1 r_1 + m_2 r_2}{m_1 + m_2}$	KAN	<u>0.0010</u>	798.5
		fKAN	<u>0.0015</u>	461.3
		PowerMLP	<u>0.0019</u>	<u>99.0</u>
		DATCNN	0.0009	86.4
I.18.4	$rF \sin(\theta)$	KAN	<u>0.0040</u>	1225.0
		fKAN	<u>0.2358</u>	232.4
		PowerMLP	<u>0.1531</u>	276.3
		DATCNN	0.0021	<u>243.9</u>
I.18.16	$mr v \sin(\theta)$	KAN	<u>0.0744</u>	1837.3
		fKAN	0.0048	494.6
		PowerMLP	<u>0.0973</u>	213.9
		DATCNN	<u>0.0058</u>	<u>370.4</u>
I.24.6	$\frac{1}{4}m(\omega^2 + \omega_0^2)x^2$	KAN	<u>0.0013</u>	322.8
		fKAN	<u>0.0020</u>	688.9
		PowerMLP	0.0012	<u>136.5</u>
		DATCNN	<u>0.0022</u>	97.5
I.25.13	$\frac{q}{C}$	KAN	<u>0.0011</u>	244.0
		fKAN	<u>0.0036</u>	384.1
		PowerMLP	<u>0.0027</u>	96.7
		DATCNN	0.0011	<u>125.7</u>

Num.	Function	Model	RMSE	Training Time(s)
I.26.2	$\arcsin(n \sin(\theta_2))$	KAN	0.0034	410.3
		fKAN	0.0068	611.6
		PowerMLP	0.0566	145.5
		DATCNN	<u>0.0037</u>	<u>297.6</u>
I.27.6	$\frac{1}{\frac{1}{d_1} + \frac{n}{d_2}}$	KAN	0.0006	743.5
		fKAN	0.1794	624.2
		PowerMLP	0.0057	<u>88.0</u>
		DATCNN	<u>0.0011</u>	44.4
I.29.4	$\frac{\omega}{c}$	KAN	0.0007	729.6
		fKAN	0.0042	500.8
		PowerMLP	0.0084	78.2
		DATCNN	<u>0.0009</u>	<u>114.0</u>
I.29.16	$\sqrt{x_1^2 + x_2^2 - 2x_1x_2 \cos(\theta_1 - \theta_2)}$	KAN	0.2056	1704.1
		fKAN	<u>0.0519</u>	519.6
		PowerMLP	0.2466	212.3
		DATCNN	0.0328	<u>444.1</u>
I.30.3	$\frac{I_0 \sin^2(\frac{n\theta}{2})}{\sin^2(\frac{\theta}{2})}$	KAN	0.0131	1602.8
		fKAN	0.3961	496.9
		PowerMLP	0.2964	144.2
		DATCNN	<u>0.0167</u>	<u>395.5</u>
I.30.5	$\arcsin(\frac{\lambda}{nd})$	KAN	0.0009	696.3
		fKAN	0.0046	469.0
		PowerMLP	0.0045	<u>144.6</u>
		DATCNN	<u>0.0011</u>	69.2
I.32.5	$\frac{q^2 a^2}{\epsilon c^3}$	KAN	<u>0.0404</u>	1034.0
		fKAN	0.3057	546.4
		PowerMLP	0.0580	<u>209.8</u>
		DATCNN	0.0062	182.4
I.32.17	$\frac{1}{2} \epsilon c E_f^2 \left(\frac{8\pi r^2}{3} \right) \frac{\omega^4}{(\omega^2 - \omega_0^2)^2}$	KAN	0.2768	1408.1
		fKAN	0.2626	424.9
		PowerMLP	<u>0.1451</u>	190.9
		DATCNN	0.0741	<u>373.4</u>
I.34.8	$\frac{qVB}{p}$	KAN	0.0083	1283.7
		fKAN	0.0126	316.4
		PowerMLP	<u>0.0044</u>	130.6
		DATCNN	0.0040	<u>255.6</u>
I.34.10	$\frac{\omega_0}{1 - \frac{v}{c}}$	KAN	0.0022	440.5
		fKAN	0.5585	595.6
		PowerMLP	<u>0.0021</u>	116.2
		DATCNN	0.0019	<u>135.8</u>
I.34.14	$\omega_0 \frac{1 + \frac{v}{c}}{\sqrt{1 - \frac{v^2}{c^2}}}$	KAN	0.0013	280.0
		fKAN	0.4018	683.7
		PowerMLP	0.0033	117.3
		DATCNN	<u>0.0014</u>	<u>126.5</u>
I.34.27	$\hbar\omega$	KAN	0.0005	755.1
		fKAN	0.0013	284.1
		PowerMLP	0.0015	61.9
		DATCNN	<u>0.0009</u>	<u>93.8</u>
I.37.4	$I_1 + I_2 + 2\sqrt{I_1 I_2} \cos(\delta)$	KAN	0.0024	646.7
		fKAN	0.8559	456.6
		PowerMLP	0.1935	161.4
		DATCNN	<u>0.0030</u>	<u>233.6</u>

Num.	Function	Model	RMSE	Training Time(s)
I.38.12	$\frac{4\pi\epsilon\hbar^2}{mq^2}$	KAN	0.1084	1614.9
		fKAN	0.8757	<u>272.1</u>
		PowerMLP	0.0981	135.0
		DATCNN	0.0369	329.8
I.39.10	$\frac{3}{2}p_F V$	KAN	0.0005	589.9
		fKAN	0.0035	197.8
		PowerMLP	0.0015	<u>112.5</u>
		DATCNN	<u>0.0008</u>	45.9
I.39.11	$\frac{p_F V}{\gamma-1}$	KAN	<u>0.0013</u>	253.2
		fKAN	0.2627	490.8
		PowerMLP	0.0041	<u>121.0</u>
		DATCNN	0.0011	74.5
I.39.22	$\frac{nk_b T}{V}$	KAN	<u>0.0012</u>	817.0
		fKAN	0.0030	391.1
		PowerMLP	0.0028	111.8
		DATCNN	0.0012	<u>163.3</u>
I.40.1	$n_0 e^{-\frac{mgx}{k_b T}}$	KAN	<u>0.0043</u>	818.1
		fKAN	0.2891	575.4
		PowerMLP	0.0064	132.9
		DATCNN	0.0033	<u>281.4</u>
I.41.16	$\frac{\hbar\omega^3}{\pi^2 c^2 \left(e^{\frac{\hbar\omega}{k_b T}} - 1 \right)}$	KAN	0.0011	256.9
		fKAN	0.0024	208.5
		PowerMLP	<u>0.0015</u>	<u>168.6</u>
		DATCNN	0.0018	59.1
I.43.16	$\frac{\mu q V_e}{d}$	KAN	<u>0.0012</u>	795.8
		fKAN	0.0030	350.6
		PowerMLP	0.0028	101.1
		DATCNN	0.0012	<u>137.8</u>
I.43.31	$\mu k_b T$	KAN	0.0007	199.9
		fKAN	0.0265	647.8
		PowerMLP	0.0022	68.1
		DATCNN	<u>0.0011</u>	<u>109.0</u>
I.43.43	$\frac{k_b v}{A(\gamma-1)}$	KAN	0.0072	347.2
		fKAN	<u>0.0062</u>	333.4
		PowerMLP	0.0225	<u>153.2</u>
		DATCNN	0.0028	91.3
I.44.4	$nk_b T \log\left(\frac{V_2}{V_1}\right)$	KAN	0.0017	310.7
		fKAN	0.0029	633.1
		PowerMLP	0.0036	160.3
		DATCNN	<u>0.0019</u>	<u>180.1</u>
I.47.23	$\sqrt{\frac{\gamma p}{\rho}}$	KAN	0.0009	746.2
		fKAN	0.2126	421.2
		PowerMLP	0.0027	<u>171.4</u>
		DATCNN	<u>0.0011</u>	136.1
I.48.20	$\frac{mc^2}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	0.0019	416.6
		fKAN	0.8881	626.7
		PowerMLP	<u>0.0030</u>	<u>232.2</u>
		DATCNN	0.0030	173.3
I.50.26	$x_1(\cos(\omega t) + \alpha \cos^2(\omega t))$	KAN	0.0456	1375.4
		fKAN	<u>0.0174</u>	637.3
		PowerMLP	0.1239	156.9
		DATCNN	0.0067	<u>352.0</u>

Num.	Function	Model	RMSE	Training Time(s)
II.2.42	$\frac{\kappa(T_2-T_1)A}{d}$	KAN	<u>0.0018</u>	317.9
		fKAN	0.0034	487.6
		PowerMLP	0.0033	77.5
		DATCNN	0.0015	<u>121.0</u>
II.3.24	$\frac{P}{4\pi r^2}$	KAN	0.0002	126.0
		fKAN	0.0050	261.6
		PowerMLP	0.0061	<u>65.9</u>
		DATCNN	<u>0.0009</u>	48.2
II.4.23	$\frac{q}{4\pi\epsilon r}$	KAN	0.0005	639.9
		fKAN	0.0010	312.2
		PowerMLP	0.0051	<u>86.1</u>
		DATCNN	<u>0.0008</u>	37.9
II.6.11	$\frac{1}{4\pi\epsilon} \frac{p_d \cos(\theta)}{r^2}$	KAN	<u>0.0035</u>	260.7
		fKAN	0.0041	372.8
		PowerMLP	0.0232	<u>219.5</u>
		DATCNN	0.0021	94.5
II.6.15a	$\frac{3}{4\pi\epsilon} \frac{p_d z}{r^5} \sqrt{x^2 + y^2}$	KAN	<u>0.0418</u>	1279.1
		fKAN	0.0680	303.4
		PowerMLP	0.0439	143.7
		DATCNN	0.0100	<u>260.2</u>
II.6.15b	$\frac{3}{4\pi\epsilon} \frac{p_d}{r^3} \cos(\theta) \sin(\theta)$	KAN	0.0171	650.3
		fKAN	<u>0.0083</u>	333.6
		PowerMLP	0.0975	101.2
		DATCNN	0.0064	<u>183.4</u>
II.8.7	$\frac{3}{5} \frac{q^2}{4\pi\epsilon d}$	KAN	0.0005	138.9
		fKAN	0.0015	253.9
		PowerMLP	0.0037	<u>83.1</u>
		DATCNN	<u>0.0007</u>	34.0
II.8.31	$\frac{1}{2} \epsilon E_f^2$	KAN	0.0002	586.5
		fKAN	0.0010	165.3
		PowerMLP	0.0009	<u>83.5</u>
		DATCNN	<u>0.0007</u>	37.8
I.10.9	$\frac{\sigma}{\epsilon(1+\chi)}$	KAN	0.0007	217.4
		fKAN	0.2146	338.2
		PowerMLP	0.0086	<u>140.8</u>
		DATCNN	<u>0.0011</u>	43.2
II.11.3	$\frac{qE_f}{m(\omega_0^2 - \omega^2)}$	KAN	0.0076	247.5
		fKAN	<u>0.0075</u>	414.5
		PowerMLP	0.0125	<u>102.2</u>
		DATCNN	0.0021	70.0
II.11.17	$n_0 \left(1 + \frac{p_d E_f \cos(\theta)}{k_b T}\right)$	KAN	0.0459	1816.0
		fKAN	<u>0.0416</u>	627.1
		PowerMLP	0.1177	142.5
		DATCNN	0.0092	<u>358.1</u>
II.11.20	$\frac{np_d^2 E_f}{3k_b T}$	KAN	0.0010	718.2
		fKAN	0.0027	272.5
		PowerMLP	<u>0.0015</u>	<u>100.0</u>
		DATCNN	0.0018	61.6
II.11.27	$\frac{n\alpha}{1-\frac{n\alpha}{3}} \epsilon E_f$	KAN	0.0191	829.4
		fKAN	0.0131	719.1
		PowerMLP	<u>0.0069</u>	114.6
		DATCNN	0.0044	<u>167.2</u>

Num.	Function	Model	RMSE	Training Time(s)
II.11.28	$1 + \frac{n\alpha}{1-\frac{n\alpha}{3}}$	KAN	0.0009	250.8
		fKAN	0.0085	427.3
		PowerMLP	0.0033	83.3
		DATCNN	<u>0.0014</u>	<u>121.5</u>
II.13.17	$\frac{1}{4\pi\epsilon c^2} \frac{2l}{r}$	KAN	0.0117	223.2
		fKAN	<u>0.0051</u>	95.2
		PowerMLP	0.0312	207.0
		DATCNN	0.0028	<u>115.4</u>
II.13.23	$\frac{\rho}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	0.0090	257.0
		fKAN	0.3535	457.3
		PowerMLP	<u>0.0040</u>	<u>86.1</u>
		DATCNN	0.0018	73.8
II.13.34	$\frac{\rho v}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	0.0051	320.5
		fKAN	0.3109	478.0
		PowerMLP	0.0030	86.0
		DATCNN	<u>0.0049</u>	<u>193.2</u>
II.15.4	$-\mu_M B \cos(\theta)$	KAN	<u>0.0019</u>	305.2
		fKAN	0.0192	391.7
		PowerMLP	0.0619	<u>214.9</u>
		DATCNN	0.0017	100.4
II.15.5	$-p_d E_f \cos(\theta)$	KAN	<u>0.0019</u>	308.9
		fKAN	0.0192	390.2
		PowerMLP	0.0619	<u>203.2</u>
		DATCNN	0.0017	98.5
II.21.32	$\frac{q}{4\pi\epsilon r(1-\frac{v}{c})}$	KAN	0.0174	243.6
		fKAN	0.0215	216.5
		PowerMLP	0.0112	98.9
		DATCNN	<u>0.0113</u>	<u>155.4</u>
II.24.17	$\sqrt{\frac{\omega^2}{c^2} - \frac{\pi^2}{d^2}}$	KAN	<u>0.0010</u>	638.9
		fKAN	0.2612	305.0
		PowerMLP	0.0094	<u>163.3</u>
		DATCNN	0.0009	120.6
II.27.16	$\epsilon c E_f^2$	KAN	0.0013	615.9
		fKAN	0.1269	149.5
		PowerMLP	0.0019	63.7
		DATCNN	<u>0.0019</u>	<u>79.7</u>
II.27.18	ϵE_f^2	KAN	0.0004	160.2
		fKAN	0.0022	311.5
		PowerMLP	0.0015	<u>64.6</u>
		DATCNN	<u>0.0010</u>	53.0
II.34.2a	$\frac{qv}{2\pi r}$	KAN	0.0006	157.5
		fKAN	0.0395	275.9
		PowerMLP	<u>0.0009</u>	<u>67.4</u>
		DATCNN	0.0010	36.3
II.34.2	$\frac{qvr}{2}$	KAN	0.0005	175.1
		fKAN	0.0724	151.0
		PowerMLP	<u>0.0012</u>	<u>67.1</u>
		DATCNN	0.0014	41.0
II.34.11	$\frac{gqB}{2m}$	KAN	<u>0.0009</u>	207.1
		fKAN	0.0022	182.3
		PowerMLP	0.0027	<u>85.3</u>
		DATCNN	0.0008	52.3

Num.	Function	Model	RMSE	Training Time(s)
II.34.29a	$\frac{qh}{4\pi m}$	KAN	0.0003	116.3
		fKAN	0.0007	206.7
		PowerMLP	<u>0.0004</u>	62.0
		DATCNN	0.0006	36.8
II.34.29b	$\frac{g\mu B J}{\hbar}$	KAN	<u>0.0019</u>	212.0
		fKAN	0.0027	310.3
		PowerMLP	0.0039	69.3
		DATCNN	0.0017	<u>85.5</u>
II.35.18	$\frac{n_0}{e^{\frac{\mu B}{k_b T}} + e^{-\frac{\mu B}{k_b T}}}$	KAN	0.0010	209.5
		fKAN	0.0034	316.4
		PowerMLP	0.0023	<u>164.0</u>
		DATCNN	<u>0.0015</u>	46.7
II.35.21	$n\mu \tanh\left(\frac{\mu B}{k_b T}\right)$	KAN	<u>0.0030</u>	821.9
		fKAN	0.0055	517.4
		PowerMLP	0.0041	117.8
		DATCNN	0.0026	<u>125.0</u>
II.36.38	$\frac{\mu B}{k_b T} + \frac{\mu \alpha M}{\epsilon c^2 k_b T}$	KAN	0.1381	1855.3
		fKAN	0.1078	501.6
		PowerMLP	<u>0.0359</u>	129.8
		DATCNN	0.0085	<u>181.8</u>
II.37.1	$\mu(1 + \chi)B$	KAN	0.0010	221.8
		fKAN	0.3009	364.4
		PowerMLP	0.0036	71.6
		DATCNN	<u>0.0014</u>	<u>89.1</u>
II.38.3	$\frac{YAx}{d}$	KAN	<u>0.0012</u>	792.7
		fKAN	0.0030	387.2
		PowerMLP	0.0028	101.8
		DATCNN	0.0012	<u>132.4</u>
II.38.14	$\frac{Y}{2(1+\sigma)}$	KAN	0.0003	607.0
		fKAN	0.0010	177.5
		PowerMLP	<u>0.0004</u>	<u>108.4</u>
		DATCNN	0.0008	37.1
III.4.32	$\frac{1}{e^{\frac{\hbar\omega}{k_b T}} - 1}$	KAN	0.0023	529.5
		fKAN	1.6992	508.2
		PowerMLP	0.0062	<u>142.1</u>
		DATCNN	<u>0.0026</u>	133.6
III.4.33	$\frac{\hbar\omega}{e^{\frac{\hbar\omega}{k_b T}} - 1}$	KAN	<u>0.0018</u>	330.9
		fKAN	0.0043	443.6
		PowerMLP	0.0037	<u>98.0</u>
		DATCNN	0.0018	97.7
III.7.38	$\frac{2\mu B}{\hbar}$	KAN	0.0022	393.4
		fKAN	0.4968	207.8
		PowerMLP	<u>0.0020</u>	110.8
		DATCNN	0.0014	<u>118.1</u>
III.8.54	$\sin^2\left(\frac{Et}{\hbar}\right)$	KAN	0.0183	896.2
		fKAN	0.0622	424.7
		PowerMLP	0.2441	211.7
		DATCNN	<u>0.0526</u>	<u>297.7</u>
III.9.52	$\frac{p_d E_f t}{\hbar} \frac{\sin^2\left(\frac{(\omega - \omega_0)t}{2}\right)}{\left(\frac{(\omega - \omega_0)t}{2}\right)^2}$	KAN	0.0025	477.2
		fKAN	0.0047	568.9
		PowerMLP	0.0150	180.4
		DATCNN	<u>0.0028</u>	<u>231.0</u>

Num.	Function	Model	RMSE	Training Time(s)
III.10.19	$\mu\sqrt{B_x^2 + B_y^2 + B_z^2}$	KAN	<u>0.0024</u>	345.8
		fKAN	0.2883	597.6
		PowerMLP	0.0032	<u>149.5</u>
		DATCNN	0.0022	118.0
III.12.43	$n\hbar$	KAN	0.0003	201.1
		fKAN	0.0019	488.3
		PowerMLP	0.0011	<u>110.5</u>
		DATCNN	<u>0.0008</u>	42.9
III.13.18	$\frac{2Ed^2k}{\hbar}$	KAN	<u>0.0022</u>	332.2
		fKAN	0.0040	363.6
		PowerMLP	0.0030	83.6
		DATCNN	0.0018	<u>138.3</u>
III.14.14	$I_0\left(e^{\frac{qV_e}{k_b T}} - 1\right)$	KAN	0.0377	340.8
		fKAN	0.0272	439.7
		PowerMLP	<u>0.0124</u>	<u>206.9</u>
		DATCNN	0.0066	167.2
III.15.12	$2U(1 - \cos(kd))$	KAN	0.0023	1155.7
		fKAN	0.8773	426.5
		PowerMLP	0.2367	139.7
		DATCNN	<u>0.0031</u>	<u>252.6</u>
III.15.14	$\frac{\hbar^2}{2Ed^2}$	KAN	<u>0.0029</u>	194.3
		fKAN	0.0056	335.2
		PowerMLP	0.0091	<u>96.5</u>
		DATCNN	0.0013	79.8
III.15.27	$\frac{2\pi\alpha}{nd}$	KAN	<u>0.0118</u>	650.8
		fKAN	2.3648	426.4
		PowerMLP	0.0744	137.9
		DATCNN	0.0021	<u>140.6</u>
III.17.37	$\beta(1 + \alpha \cos(\theta))$	KAN	<u>0.0024</u>	802.0
		fKAN	0.1433	464.5
		PowerMLP	0.0395	<u>179.8</u>
		DATCNN	0.0018	159.2
III.19.51	$-\frac{mq^4}{2(4\pi\epsilon)^2\hbar^2} \frac{1}{n^2}$	KAN	0.0008	119.0
		fKAN	0.0013	247.3
		PowerMLP	<u>0.0011</u>	<u>87.9</u>
		DATCNN	0.0012	32.5
III.21.20	$-\frac{\rho q A}{m}$	KAN	0.0013	673.7
		fKAN	0.0084	209.8
		PowerMLP	0.0032	<u>102.5</u>
		DATCNN	<u>0.0013</u>	75.4
Rutherfordd scattering	$\left(\frac{Z_1 Z_2 \alpha \hbar c}{4E \sin^2(\frac{\theta}{2})}\right)^2$	KAN	0.1286	1348.5
		fKAN	0.1303	510.9
		PowerMLP	<u>0.0896</u>	<u>140.7</u>
		DATCNN	0.0126	129.0
Friedman equation	$\sqrt{\frac{8\pi G}{3}\rho - \frac{k_f c^2}{a_f^2}}$	KAN	<u>0.0023</u>	891.6
		fKAN	0.5975	604.4
		PowerMLP	0.0040	98.0
		DATCNN	0.0015	<u>106.0</u>
Compton scattering	$\frac{E}{1 + \frac{E}{mc^2}(1 - \cos(\theta))}$	KAN	<u>0.0032</u>	548.4
		fKAN	0.0033	701.1
		PowerMLP	0.0557	<u>225.6</u>
		DATCNN	0.0024	141.3

Num.	Function	Model	RMSE	Training Time(s)
Relativistic aberration	$\arccos\left(\frac{\cos(\theta_2) - \frac{v}{c}}{1 - \frac{v}{c}\cos(\theta_2)}\right)$	KAN	<u>0.0073</u>	375.9
		fKAN	0.9117	398.9
		PowerMLP	0.0106	<u>157.8</u>
		DATCNN	0.0045	102.5
N-slit diffraction	$I_0 \left(\frac{\sin(\frac{\alpha}{2})}{\frac{\alpha}{2}} \frac{\sin(\frac{N\delta}{2})}{\sin(\frac{\delta}{2})} \right)^2$	KAN	0.0012	263.9
		fKAN	0.0019	234.5
		PowerMLP	0.0020	<u>126.7</u>
		DATCNN	<u>0.0015</u>	53.0
Goldstein 3.16	$\sqrt{\frac{2}{m} \left(E - U - \frac{L^2}{2mr^2} \right)}$	KAN	0.0012	321.5
		fKAN	0.2337	594.2
		PowerMLP	0.0023	<u>157.3</u>
		DATCNN	<u>0.0014</u>	56.5
Goldstein 3.55	$\frac{mk_G}{L^2} \left(1 + \sqrt{1 + \frac{2EL^2}{mk_G^2}} \cos(\theta_1 - \theta_2) \right)$	KAN	<u>0.1341</u>	2217.7
		fKAN	2.6119	<u>338.6</u>
		PowerMLP	1.6879	142.6
		DATCNN	0.0492	445.7
Goldstein 3.64 (ellipse)	$\frac{d(1-\alpha^2)}{1+\alpha\cos(\theta_2-\theta_1)}$	KAN	<u>0.0166</u>	1364.0
		fKAN	0.0218	578.2
		PowerMLP	0.1140	139.1
		DATCNN	0.0048	<u>264.9</u>
Goldstein 3.74 (Kepler)	$\frac{2\pi d^{\frac{3}{2}}}{\sqrt{G(m_1+m_2)}}$	KAN	0.0024	782.3
		fKAN	1.2260	241.5
		PowerMLP	0.0123	89.2
		DATCNN	<u>0.0031</u>	<u>129.8</u>
Goldstein 3.99	$\sqrt{1 + \frac{2\epsilon^2 EL^2}{m(Z_1 Z_2 q^2)^2}}$	KAN	<u>0.0365</u>	470.0
		fKAN	0.2136	108.4
		PowerMLP	0.0633	<u>134.6</u>
		DATCNN	0.0057	140.3
Goldstein 8.56	$\sqrt{(p - qA)^2 c^2 + m^2 c^4} + qV_e$	KAN	<u>0.0096</u>	1266.1
		fKAN	0.3116	433.0
		PowerMLP	0.0192	146.5
		DATCNN	0.0079	<u>358.5</u>
Goldstein 12.80	$\frac{1}{2m} (p^2 + m^2 \omega^2 x^2 (1 + \frac{\alpha y}{x}))$	KAN	0.0099	476.3
		fKAN	0.2102	371.5
		PowerMLP	<u>0.0069</u>	101.5
		DATCNN	0.0042	<u>126.6</u>
Jackson 2.11	$\frac{q}{4\pi\epsilon y^2} \left(4\pi\epsilon V_e d - \frac{qdy^3}{(y^2-d^2)^2} \right)$	KAN	<u>0.0390</u>	386.1
		fKAN	0.0705	275.7
		PowerMLP	0.0573	96.4
		DATCNN	0.0237	<u>192.3</u>
Jackson 4.60	$E_f \cos(\theta) \left(\frac{\alpha-1}{\alpha+2} \frac{d^3}{r^2} - r \right)$	KAN	0.0181	1164.5
		fKAN	0.0280	531.0
		PowerMLP	<u>0.0131</u>	201.4
		DATCNN	0.0110	<u>280.7</u>
Jackson 11.38 (Doppler)	$\frac{\sqrt{1 - \frac{v^2}{c^2}}}{1 + \frac{v}{c} \cos(\theta)} \omega$	KAN	0.0292	883.9
		fKAN	<u>0.0099</u>	621.6
		PowerMLP	0.0114	244.1
		DATCNN	0.0052	<u>359.2</u>
Weinberg 15.2.1	$\frac{3}{8\pi G} \left(\frac{c^2 k_f}{a_f^2} + H^2 \right)$	KAN	0.0021	<u>284.9</u>
		fKAN	0.0037	507.6
		PowerMLP	<u>0.0020</u>	170.5
		DATCNN	0.0020	296.9

Num.	Function	Model	RMSE	Training Time(s)
Weinberg 15.2.2	$-\frac{1}{8\pi G} \left(\frac{c^4 k_f}{a_f^2} + c^2 H^2 (1 - 2\alpha) \right)$	KAN	<u>0.0028</u>	<u>249.1</u>
		fKAN	0.0028	373.6
		PowerMLP	0.0029	90.0
		DATCNN	0.0022	281.9

Table A.11: Result of Special Function Dataset

Name	Function	Model	RMSE	Training Time(s)
Jacobian elliptic functions	$(\text{sn}, \text{cn}, \text{dn}, \phi)(u, m)$	KAN	0.0005	307.1
		fKAN	0.0660	819.6
		PowerMLP	0.0025	<u>71.8</u>
		DATCNN	<u>0.0010</u>	52.8
Incomplete elliptic integral of the first kind	$\int_0^\phi \frac{d\theta}{\sqrt{1-m\sin^2\theta}}$	KAN	<u>0.0826</u>	1252.3
		fKAN	1.8214	<u>368.8</u>
		PowerMLP	0.3356	142.1
		DATCNN	0.0305	474.2
Incomplete elliptic integral of the second kind	$\int_0^\phi \sqrt{1-m\sin^2\theta} d\theta$	KAN	0.0016	735.6
		fKAN	1.5523	428.8
		PowerMLP	0.0816	85.6
		DATCNN	<u>0.0033</u>	<u>177.4</u>
Bessel function of the first kind	$J_n(x)$	KAN	0.0033	288.3
		fKAN	0.0128	614.5
		PowerMLP	0.0364	<u>181.7</u>
		DATCNN	<u>0.0049</u>	153.5
Modified Bessel function of the first kind	$I_n(x)$	KAN	0.0052	197.7
		fKAN	0.0081	450.9
		PowerMLP	0.0216	<u>166.6</u>
		DATCNN	<u>0.0055</u>	166.5
Associated Legendre function ($m = 0$)	$P_l^0(x)$	KAN	<u>0.0117</u>	397.3
		fKAN	0.0482	488.5
		PowerMLP	0.0177	123.6
		DATCNN	0.0073	<u>197.3</u>
Associated Legendre function ($m = 1$)	$P_l^1(x)$	KAN	0.0334	1026.7
		fKAN	0.0885	621.2
		PowerMLP	0.1188	125.6
		DATCNN	<u>0.0434</u>	<u>405.6</u>
spherical harmonics ($m = 0, n = 1$)	$Y_1^0(\theta, \phi)$	KAN	0.0004	158.4
		fKAN	0.0010	326.3
		PowerMLP	0.0407	<u>142.7</u>
		DATCNN	<u>0.0008</u>	58.4
spherical harmonics ($m = 1, n = 1$)	$Y_1^1(\theta, \phi)$	KAN	0.0009	222.2
		fKAN	0.0014	326.7
		PowerMLP	0.0629	<u>137.3</u>
		DATCNN	<u>0.0011</u>	89.3
spherical harmonics ($m = 0, n = 2$)	$Y_2^0(\theta, \phi)$	KAN	0.0005	179.9
		fKAN	0.0026	327.3
		PowerMLP	0.0232	<u>133.7</u>
		DATCNN	<u>0.0008</u>	51.6
spherical harmonics ($m = 1, n = 2$)	$Y_2^1(\theta, \phi)$	KAN	0.0011	279.4
		fKAN	0.0022	441.9
		PowerMLP	0.1226	86.3
		DATCNN	<u>0.0018</u>	<u>114.4</u>

Num.	Function	Model	RMSE	Training Time(s)
spherical harmonics ($m = 2, n = 2$)	$Y_2^2(\theta, \phi)$	KAN	0.0018	336.7
		fKAN	<u>0.0019</u>	474.3
		PowerMLP	0.1312	92.8
		DATCNN	0.0019	<u>147.8</u>

Journal Pre-proof

References

- [1] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, *Neural networks* 2 (5) (1989) 359–366.
- [2] S. Haykin, *Neural networks: a comprehensive foundation*, Prentice Hall PTR, 1998.
- [3] G. Cybenko, Approximation by superpositions of a sigmoidal function, *Mathematics of control, signals and systems* 2 (4) (1989) 303–314.
- [4] A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, *Advances in neural information processing systems* 25.
- [5] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [6] A. Vaswani, Attention is all you need, *Advances in Neural Information Processing Systems*.
- [7] Y. Zhang, P. Tiño, A. Leonardis, K. Tang, A survey on neural network interpretability, *IEEE Transactions on Emerging Topics in Computational Intelligence* 5 (5) (2021) 726–742.
- [8] R. Caruana, H. Kangaroo, J. D. Dionisio, U. Sinha, D. Johnson, Case-based explanation of non-case-based learning methods, in: *Proceedings of the AMIA Symposium*, 1999, p. 212.
- [9] J. Bien, R. Tibshirani, Prototype selection for interpretable classification.
- [10] K. Simonyan, A. Vedaldi, A. Zisserman, Deep inside convolutional networks: Visualising image classification models and saliency maps, *arXiv preprint arXiv:1312.6034*.
- [11] M. D. Zeiler, R. Fergus, Visualizing and understanding convolutional networks, in: *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part I 13*, Springer, 2014, pp. 818–833.
- [12] J. T. Springenberg, A. Dosovitskiy, T. Brox, M. Riedmiller, Striving for simplicity: The all convolutional net, *arXiv preprint arXiv:1412.6806*.
- [13] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra, Grad-cam: Visual explanations from deep networks via gradient-based localization, in: *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 618–626.
- [14] D. Erhan, Y. Bengio, A. Courville, P. Vincent, Visualizing higher-layer features of a deep network, *University of Montreal* 1341 (3) (2009) 1.
- [15] Y. Wang, H. Su, B. Zhang, X. Hu, Interpret neural networks by identifying critical data routing paths, in: *proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8906–8914.
- [16] R. Setiono, H. Liu, Understanding neural networks via rule extraction, in: *IJCAI, Vol. 1*, Citeseer, 1995, pp. 480–485.
- [17] N. Makke, S. Chawla, Interpretable scientific discovery with symbolic regression: a review, *Artificial Intelligence Review* 57 (1) (2024) 2.
- [18] S.-M. Udrescu, M. Tegmark, Ai feynman: A physics-inspired method for symbolic regression, *Science advances* 6 (16) (2020) eaay2631.
- [19] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljacic, T. Y. Hou, M. Tegmark, KAN: Kolmogorov–arnold networks, in: *The Thirteenth International Conference on Learning Representations*, 2025.
- [20] A. A. Aghaei, fkan: Fractional kolmogorov–arnold networks with trainable jacobi basis functions, *Neurocomputing* 623 (2025) 129414.
- [21] A. A. Aghaei, rkan: Rational kolmogorov–arnold networks, *arXiv preprint arXiv:2406.14495*.
- [22] S. SS, K. AR, A. KP, et al., Chebyshev polynomial-based kolmogorov–arnold networks: An efficient architecture for nonlinear function approximation, *arXiv preprint arXiv:2405.07200*.
- [23] R. Qiu, Y. Miao, S. Wang, Y. Zhu, L. Yu, X.-S. Gao, Powermlp: An efficient version of kan, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, 2025, pp. 20069–20076.
- [24] C. Li, X. Liu, W. Li, C. Wang, H. Liu, Y. Liu, Z. Chen, Y. Yuan, U-kan makes strong backbone for medical image segmentation and generation, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, 2025, pp. 4652–4660.
- [25] C. Cen, F. Li, P. Hu, Z. Li, A multi-scale vision mixture-of-experts for salient object detection with kolmogorov–arnold adapter, *Neurocomputing* (2025) 130349.
- [26] A. D. Bodner, A. S. Tepsich, J. N. Spolski, S. Pourteau, Convolutional kolmogorov–arnold networks, *arXiv preprint arXiv:2406.13155*.
- [27] R. Genet, H. Inzirillo, Tkan: Temporal kolmogorov–arnold networks, *arXiv preprint arXiv:2405.07344*.
- [28] R. Bresson, G. Nikolentzos, G. Panagopoulos, M. Chatzianastasis, J. Pang, M. Vazirgiannis, Kagnns: Kolmogorov–arnold networks meet graph learning, *arXiv preprint arXiv:2406.18380*.
- [29] M. Taghizadeh, Z. Zandsalimi, M. A. Nabian, M. Shafiee-Jood, N. Alemazkoo, Interpretable physics-informed graph neural networks for flood forecasting, *Computer-Aided Civil and Infrastructure Engineering*.
- [30] D. Fakhoury, E. Fakhoury, H. Speleers, Exspline: An interpretable and expressive spline-based neural network, *Neural Networks* 152 (2022) 332–346.
- [31] H. Xiao, K. Rasul, R. Vollgraf, Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms, *arXiv preprint arXiv:1708.07747*.
- [32] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, A. Y. Ng, et al., Reading digits in natural images with unsupervised feature learning, in: *NIPS workshop on deep learning and unsupervised feature learning*, Vol. 2011, Granada, 2011, p. 7.
- [33] X. Li, D. Roth, Learning question classifiers, in: *COLING 2002: The 19th International Conference on Computational Linguistics*, 2002.
- [34] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, L. Specia, Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation, *arXiv preprint arXiv:1708.00055*.
- [35] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, S. Bowman, Glue: A multi-task benchmark and analysis platform for natural language understanding, in: *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP*, 2018, pp. 353–355.
- [36] E. M. Voorhees, H. T. Dang, Overview of the trec 2001 question answering track., in: *Trec*, 2001, pp. 42–51.
- [37] J. Pennington, R. Socher, C. D. Manning, Glove: Global vectors for word representation, in: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1532–1543.
- [38] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, *arXiv preprint arXiv:1908.10084*.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.